اونيۏرسيتي مليسيا قهڠ السلطان عبد الله
UNIVERSITI MALAYSIA PAHANG
AL-SULTAN ABDULLAH

FACULTY OF COMPUTING

SEMESTER I ACADEMIC SESSION 2025/2026

BLOCKCHAIN TECHNOLOGY (BCI3353)

FINAL ASSESSMENT

PROJECT REPORT

TITLE

Blockchain-Based Academic Credential Verification System

STUDENT NAME: MUHAMMAD SHAHRIZUAN BIN MOHD ROMZI

MATRIC ID: CA23085

PROGRAM: BACHELOR OF COMPUTER SCIENCE (COMPUTER SYSTEMS & NETWORKING) WITH HONOURS

TABLE OF CONTENT

CRITERIA 1: PROJECT PROPOSAL & SYSTEM DESIGN

1.1 Problem Analysis

1.1.1 Background of the problem

Today, academic credentials like degrees and certificates are very important for getting jobs and continuing education. However, the current way of verifying these credentials has many problems. Most universities still use paper certificates or store records in their own databases. When an employer wants to check if someone really has a degree, they have to contact the university directly, which takes a very long time, sometimes weeks or even months. Also, there are many cases of fake degrees being sold online, making it hard to tell which certificates are real. I believe using blockchain technology can help solve these problems and make the verification process faster, safer, and more reliable.

1.1.2 Problem Description

The main problem is that verifying academic credentials is difficult, slow, and not secure enough. The current system has several major issues. First, the verification process takes too much time because employers must send emails or letters to universities and wait for replies. Second, fake certificates are everywhere and very hard for employers to detect. Third, students don't have control over their own credentials - they must always ask the university to send verification letters. Fourth, cross-border verification is even more complicated because different countries have different systems. Lastly, all records are kept in centralized databases that can be hacked, modified, or lost. If something happens to the university's database, all student records could be gone.

1.1.3 Stakeholders Involved

- Students/Graduates: They are the owners of their academic credentials and need to share them with employers or other universities when applying for jobs or further studies. Currently, students face difficulties when they need quick verification. With blockchain, they can have better control over their own credentials.
- Educational Institutions: Universities and colleges issue degrees and certificates. Right now, they spend a lot of time and money answering verification requests from employers. Many universities receive hundreds of requests every month,

creating lots of administrative work. They want a system that reduces this workload while keeping credentials secure.

- Employers: Companies need to verify if job candidates really have the qualifications they claim. They want a fast and reliable way to check credentials without waiting weeks. Employers also want to avoid hiring people with fake degrees.
- Government Regulatory Bodies: Education ministries and accreditation agencies oversee education quality and ensure only legitimate institutions can issue valid credentials. They need to monitor credential issuance and maintain standards.
- Verification Service Providers: Third-party companies that provide verification services. With blockchain, they can offer additional services and help employers verify credentials more efficiently.

1.1.4 Limitation of Current System

- Very Slow Verification Process: Verification typically takes 2-6 weeks or longer for international credentials. This delay causes problems for students who need to start work quickly or meet deadlines.
- Easy to Fake Credentials: Paper certificates can be forged using design software and printers. Even digital records can be tampered with if someone accesses the university database. Credential fraud is a growing problem.
- Expensive Administrative Costs: Universities spend money maintaining databases and paying staff to process verification requests. They often charge students fees for each verification, which adds up quickly.
- Students Don't Control Their Own Data: Students depend completely on universities to access their records. This becomes a problem if the university closes or has outdated systems.
- No Standard Format: Different universities use different formats (paper, PDFs, online portals), making it difficult to create unified systems, especially internationally.
- Privacy Issues: Students must share entire transcripts even when employers only need basic information like graduation status, exposing unnecessary personal data.

1.1.5 Justification for Blockchain Adoption

I believe blockchain technology is perfect for solving these problems because it has special features that address all the issues mentioned above.

- Immutability: Once a credential is recorded on blockchain, it cannot be changed or deleted. When a university issues a degree, nobody can fake or modify it later. This completely solves the fake certificate problem.
- Decentralization: Instead of keeping records in one place, blockchain stores copies across many computers. Even if a university's system crashes or closes down, the credentials are still safe and accessible.
- Transparency and Trust: Blockchain allows everyone to verify credentials independently without third parties. The process is transparent, creating trust while protecting privacy through authorized access only.
- Traceability: Every credential has a complete history showing exactly when it was issued, by which institution, and to which student. This audit trail makes fraud impossible.
- Smart Contract Automation: Smart contracts verify credentials instantly without human involvement. Instead of waiting weeks, employers get results in seconds while reducing errors and costs.
- Student Ownership: Students get digital wallets with their credentials and private keys, giving them complete control. They can share credentials directly without asking the university and choose to share only specific information.

1.1.6 Brief explanation of blockchain-based solution

I will develop a Blockchain-Based Academic Credential Verification System using Hyperledger Fabric, a permissioned blockchain suitable for academic credentials because only authorized members can participate. Universities will be able to issue credentials directly onto the blockchain. When a student graduates, the university creates a digital credential with information like degree name, graduation date, and student's name. This credential is recorded on the blockchain with a unique ID and cryptographic hash. Once recorded, it cannot be changed or deleted.

Students will receive a digital wallet containing all their credentials with private keys giving them full control. When applying for a job and the employer wants verification, the student gives permission for the employer to check the credential on the blockchain. The employer can then instantly verify if the credential is real. Smart contracts will manage the whole process including credential issuance, verification, and revocation. For example, only authorized universities can issue credentials, and only people with permission can verify them. Students can choose what information to share - if an employer only needs to know about degree completion, the student can share just that without showing complete transcripts.

The system keeps detailed logs of all verification requests for accountability. For international students, this is especially helpful because credentials from different countries can all be stored in the same blockchain network, making cross-border verification much easier. My solution also integrates with existing university systems. Universities don't have to completely replace their current systems. When a university issues a degree in their regular system, it can automatically create a blockchain record too.

Overall, this blockchain solution will transform academic credential verification from a slow, expensive, and risky process into something that is instant, secure, affordable, and student-controlled.

1.2 System Architecture Design

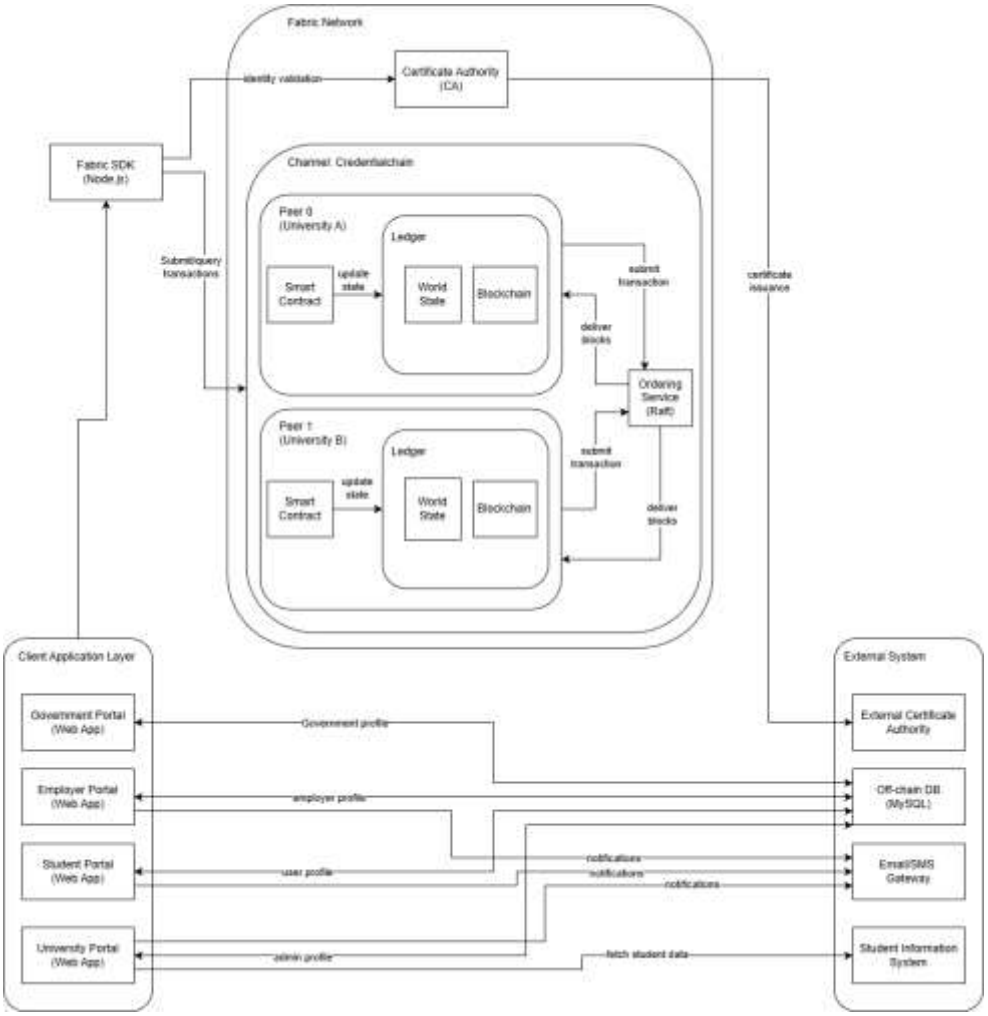


Figure 1: Diagram System Architecture Design

Explanation:

1. Certificate Authority (CA): The CA manages all identities in the network. When a university or user joins, the CA gives them digital certificates that prove who they are. These certificates control who can do what in the system.
2. Peers: Peers are computers that store blockchain data and run smart contracts. Each university will have at least one peer node. These peers keep copies of all credential records. When someone verifies a credential, the peer checks the smart contract and blockchain data. Having multiple peers makes the system more reliable.
3. Orderers: The orderer organizes transactions into blocks. When multiple transactions happen simultaneously, the orderer collects them, puts them in order, creates a new block, and sends it to all peers. I will use Raft ordering service.

4. Channel (CredentialChain): A channel is like a private room where only authorized members can see what's happening. I created a channel called "CredentialChain" specifically for academic credentials to keep data private and secure.
5. Smart Contract (Chaincode): The smart contract contains all the business logic. It's the code that defines how credentials are issued, verified, and revoked. I will write this in JavaScript. It runs on peers and processes transactions according to rules I set.
6. Ledger: The ledger stores all data in two parts. The World State stores current status of all credentials (like a database). The Blockchain records every transaction that ever happened. Once written to blockchain, it cannot be changed.
7. Client Application Layer: This is the user interface people interact with. I will create different web portals for students, universities, employers, and government. All portals connect to the blockchain through Fabric SDK.

1.3 Data Transaction Flow Design

1.3.1 Entity Attribute Table

Table 1: Credential Entity

Attribute	Data Type	Description
credentialID	String	Unique identifier (Primary Key)
studentID	String	Links to student who owns this credential
institutionID	String	Links to issuing university
credentialType	String	Type: Degree, Diploma, Certificate, Transcript
programName	String	Program name such as "Bachelor of Computer Science"
major	String	Field of study or specialization
dateIssued	DateTime	Date when credential was issued
dateCompleted	DateTime	Date when program was completed
grade	String	Final grade or GPA
certificateHash	String	Cryptographic hash for authenticity
status	String	Current status: Active, Revoked, Suspended
metadata	JSON	Extra information like honors, awards

Table 2: Student Entity

Attribute	Data Type	Description
studentID	String	Unique identifier (Primary Key)
firstName	String	Student's first name
lastName	String	Student's last name
dateOfBirth	Date	Student's birth date
email	String	Student's email address
publicKey	String	Blockchain public key for verification
enrollmentDate	DateTime	When student record was created

Table 3: Institution Entity

Attribute	Data Type	Description
institutionID	String	Unique identifier (Primary Key)
institutionName	String	Official university name
country	String	Country location
accreditationBody	String	Accrediting organization
publicKey	String	University's blockchain key
status	String	Active, Inactive, or Suspended
registrationDate	DateTime	When university joined network

Table 4: VerificationRequest Entity

Attribute	Data Type	Description
requestID	String	Unique identifier (Primary Key)
credentialID	String	Which credential is being verified
requesterID	String	Who is requesting (employer/institution ID)
requesterType	String	Type: Employer, Institution, Individual
requestDate	DateTime	When verification was requested
verificationResult	Boolean	Result: true (valid) or false (invalid)
accessGranted	Boolean	Did student give permission?
expiryDate	DateTime	When verification access expires

1.3.2 State Transition Diagram

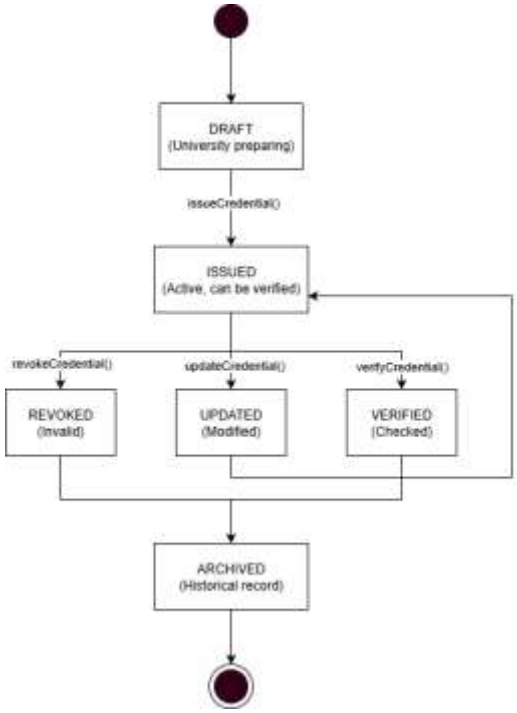


Figure 2: State Transition Diagram

1.3.3 State Transition Table

Current State	Transaction	Next State	Conditions	Authorized By
DRAFT	issueCredential()	ISSUED	All fields complete, valid student ID, authorized issuer	Institution
ISSUED	verifyCredential()	VERIFIED	Valid credential ID, authorized verifier	Student/Institution/ Employer
ISSUED	updateCredential()	UPDATED	Valid update reason, authorized by issuer	Institution
ISSUED	revokeCredential()	REVOKED	Valid revocation reason (fraud, error)	Institution/Regulator
REVOKED	reinstateCredential()	ISSUED	Issue resolved, approved by authority	Institution/Regulator
UPDATED	verifyCredential()	VERIFIED	Updated credential valid	Authorized parties
Any State	archiveCredential()	ARCHIVED	Old record (>10 years)	System/Institution

1.4 Smart Contract Design Specification

1.4.1 Function 1: issueCredential

Purpose:

- Allows universities to create new credentials and record them on the blockchain. When a student graduates, the university uses this function to issue their degree certificate.

Input Parameter:

- `credentialID` (String): Unique code like "CRED2025001"

- `studentID` (String): Student's ID

- `institutionID` (String): University's ID

- `credentialType` (String): Degree, Diploma, Certificate, or Transcript

- `programName` (String): such as "Bachelor of Computer Science"

- `major` (String): Field of study

- `dateIssued` (DateTime): Issuance date

- `dateCompleted` (DateTime): Program completion date

- `grade` (String): Final grade like "3.75 CGPA"

- `certificateHash` (String): Cryptographic hash of certificate

- `metadata` (JSON): Extra information like awards

Processing Logic:

1. Verify caller is a registered university using MSP
2. Check credentialID doesn't already exist (must be unique)
3. Validate studentID exists in system
4. Ensure institutionID matches caller's organization
5. Validate all required fields are filled correctly
6. Check dates are logical (completion before issue date)
7. Create credential object with status="ISSUED"
8. Store credential using putState()
9. Create event "CredentialIssued"
10. Log transaction with timestamp

Output/Result:

- Success: Returns credential details and transaction ID, emits event with credentialID, studentID, institutionID, timestamp
- Failure: Returns error message ("Credential ID already exists", "Unauthorized issuer", "Invalid student ID")

1.4.2 Function 2: verifyCredential

Purpose:

- Lets employers and institutions check if a credential is real and valid. This prevents hiring people with fake degrees.

Input Parameter:

- `credentialID` (String): Credential ID to verify

- `requesterID` (String): ID of entity requesting verification

- `requesterType` (String): Employer, Institution, or Individual

- `studentConsent` (Boolean): Student permission (true/false)

Processing Logic:

1. Verify caller is registered employer/institution

2. Check studentConsent is true (privacy requirement)

3. Query blockchain using getState() with credentialID

4. Verify credential exists

5. Check status is "ISSUED" or "VERIFIED" (not REVOKED)

6. Validate certificate hash (integrity check)

7. Create verification request record

8. Log verification with timestamp

9. Create event "CredentialVerified"

Output/Result:

- Success: Returns verification result (true/false), credential details (type, program, institution, date, grade), verification timestamp, transaction ID
- Failure: Returns error ("Credential not found", "Credential revoked", "Unauthorized access", "Student consent required")
- Event: `CredentialVerified(credentialID, requesterID, verificationResult, timestamp)`

1.4.3 Function 3: revokeCredential

Purpose:

- Allows universities or regulators to cancel a credential due to fraud, academic misconduct, or errors. Once revoked, it will fail all future verifications.

Input Parameter:

- `credentialID` (String): Credential ID to revoke

- `revokerID` (String): Entity performing revocation

- `revocationReason` (String): Fraud, Error, Misconduct, or Other

- `revocationDetails` (String): Detailed explanation

- `revokedBy` (String): Name/title of authorized person

Processing Logic:

1. Verify caller is issuing institution OR regulatory authority

2. Check revokerID matches institutionID or has regulatory authority

3. Query blockchain to get credential

4. Verify credential exists and is currently ISSUED or VERIFIED

5. Validate revocationReason is from approved list

6. Update status to "REVOKED"

7. Add revocation metadata (date, revokedBy, reason, details)

8. Update credential using putState()

9. Store separate revocation record for audit trail

10. Create event "CredentialRevoked"

Output/Result:

- Success: Returns updated credential with status "REVOKED", revocation details, transaction ID
- Failure: Returns error ("Credential not found", "Unauthorized revocation", "Already revoked")
- Event: `CredentialRevoked(credentialID, revokerID, revocationReason, timestamp)`

1.4.4 Function 4: queryCredentialsByStudent

Purpose:

- Retrieves all credentials for a specific student, allowing students to view their complete academic record and enabling portfolio displays with proper authorization.

Input Parameter:

- `studentID` (String): Student's unique identifier

- `requesterID` (String): ID of entity requesting query

- `includeRevoked` (Boolean): Include/exclude revoked credentials (default: false)

Processing Logic:

1. Verify caller authorization (must be student, their institution, or authorized verifier)

2. Check if requesterID matches studentID OR has proper authorization

3. Execute range query on blockchain to find all credentials for student

4. Filter results based on includeRevoked flag

5. Retrieve associated institution details for each credential

6. Sort credentials by dateIssued (most recent first)

7. Calculate statistics (total credentials, active count, revoked count)

8. Apply privacy filters based on requester type

9. Create query log entry for audit

Output/Result:

- Success: Returns JSON array with list of credentials (ID, type, program, institution, date, grade, status), summary statistics, student info, query timestamp
- Failure: Returns error ("Student not found", "Unauthorized access", "No credentials found")
- Event: `CredentialsQueried(studentID, requesterID, resultCount, timestamp)`

1.4.5 Summary Table

Function	Purpose	Key Inputs	Key Outputs	Access Control
issueCredential	Create new credential	credentialID, studentID, program details	Credential object, txID	Institution only
verifyCredential	Authenticate validity	credentialID, requesterID, consent	Verification result, credential data	Employers, Institutions with consent
revokeCredential	Invalidate credential	credentialID, revocationReason	Updated credential (REVOKED status)	Institution, Regulators
queryCredentialsByStudent	Retrieve student credentials	studentID, requesterID	Array of credentials, statistics	Student, Authorized parties

CRITERIA 2: SMART CONTACT IMPLEMENTATION

2.1 Develop a working smart contract

I developed a working smart contract (chaincode) written in JavaScript (Node.js) that defines and executes the business logic of the Academic Credential Verification System. The smart contract includes the following components:

Main Contract File: credentialContract.js

1. issueCredential()
   - Allows universities to issue new academic credentials with full validation including:
     - Duplicate credential ID checking
     - Student existence verification
     - Institution authorization
     - Date validation
     - Automatic status setting to "ISSUED"

2. verifyCredential()
   - Enables employers to verify credential authenticity with:
     - Credential existence checking
     - Student consent verification
     - Revocation status checking
     - Verification history logging
     - Status update to "VERIFIED"

3. revokeCredential()
   - Allows authorized institutions to revoke credentials with:
     - Authorization checking (only issuing institution can revoke)
     - Reason validation
     - Complete revocation metadata recording
     - Status update to "REVOKED"

4. queryCredentialsByStudent()
    - Retrieves all credentials for a specific student with:
        - Student existence validation
        - Filtering options for revoked credentials
        - Sorting by issue date
        - Statistical calculation

Validation and Error Handling:

- The smart contract includes comprehensive validation checks:
    - Input validation for all parameters
    - Existence checks for credentials, students, and institutions
    - Authorization verification
    - Status validation before state changes
    - Detailed error messages for failed transactions

Helper Functions:

- I also implemented helper functions for testing purposes:
    - registerStudent() - Registers new students in the system
    - registerInstitution() - Registers new universities
    - _readLedger() and _writeLedger() - Handle file operations

Step 1: Make a directory of credential-verification-system and enter that directory.

```
C:\Users\Admin>mkdir credential-verification-system

C:\Users\Admin>cd credential-verification-system

C:\Users\Admin\credential-verification-system>
```

Figure 3: Step 1

Step 2: used the npm init -y command to automatically generate a package.json file

```
C:\Users\Admin\credential-verification-system>npm init -y
Wrote to C:\Users\Admin\credential-verification-system\package.json:

{
  "name": "credential-verification-system",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}



C:\Users\Admin\credential-verification-system>
```

Figure 4: Step 2

Step 3: ran the command npm install fs-extra express cors to download and install the specific libraries needed for the project's server and file handling.

```
C:\Users\Admin\credential-verification-system>npm install fs-extra express c
ors

added 71 packages, and audited 72 packages in 4s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Admin\credential-verification-system>
```

Figure 5: Step 3

Step 4: Make a directory inside the credential-verification-system which is directory contracts, simulation, ledger and web.

```
C:\Users\Admin\credential-verification-system>mkdir contracts

C:\Users\Admin\credential-verification-system>mkdir simulation

C:\Users\Admin\credential-verification-system>mkdir ledger

C:\Users\Admin\credential-verification-system>mkdir web

C:\Users\Admin\credential-verification-system>
```

Figure 6: Step 4

Step 5: create a new file named ledger.json inside the ledger folder and filled it with empty curly braces {} to serve as my initial database.

```
C:\Users\Admin\Documents\credential-verification-system>echo {} > ledger\led
ger.json
```

Figure 7: Step 5

Step 6: To check the directory after creating is exist or not.

```
C:\Users\Admin\Documents\credential-verification-system>dir
 Volume in drive C is Windows-SSD
 Volume Serial Number is 92F2-7015

 Directory of C:\Users\Admin\Documents\credential-verification-system

04/01/2026  02:45 PM    <DIR>          .
04/01/2026  02:43 PM    <DIR>          ..
04/01/2026  02:44 PM    <DIR>          contracts
04/01/2026  02:45 PM    <DIR>          ledger
04/01/2026  02:44 PM    <DIR>          node_modules
04/01/2026  02:44 PM            31,359 package-lock.json
04/01/2026  02:44 PM               393 package.json
04/01/2026  02:45 PM    <DIR>          simulation
04/01/2026  02:45 PM    <DIR>          web
               2 File(s)         31,752 bytes
               7 Dir(s)  54,643,576,832 bytes free

C:\Users\Admin\Documents\credential-verification-system>
```

Figure 8: Step 6

2.1.1 CREATE OPERATION

Function Name: issueCredential()

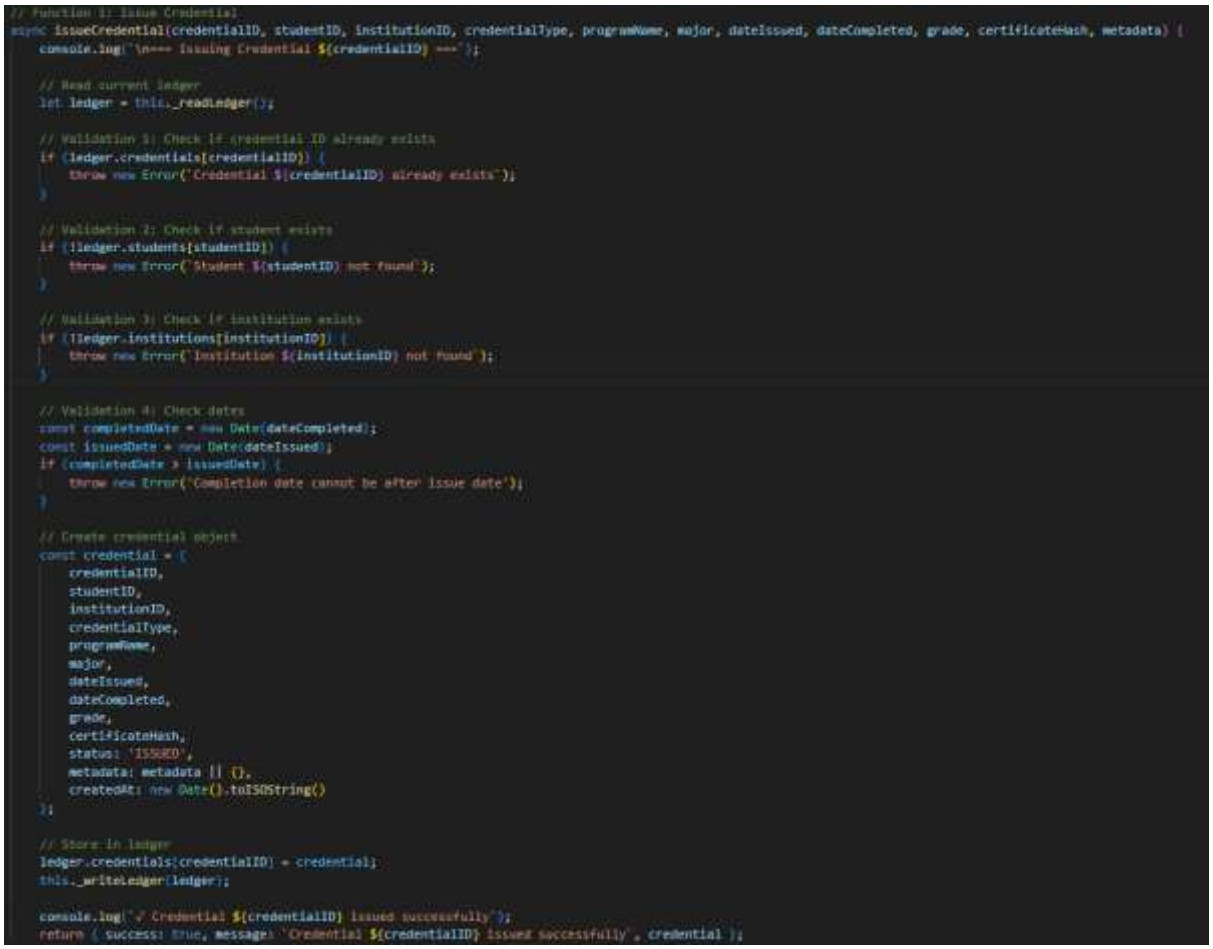


Figure 9: Create Operation code

CREATE Operation – issueCredential()

- This function creates new academic credentials in the blockchain. It performs validation checks for duplicate IDs, student existence, institution authorization, and date logic before storing the credential in the ledger with status "ISSUED".

2.1.2 READ OPERATION

Function Name: queryCredentialsByStudent()

```
// Function 4: Query Credentials by Student
async queryCredentialsByStudent(studentID, requesterID, includeRevoked = false) {
    console.log(`\n=== Querying Credentials for Student ${studentID} ===`);

    let ledger = this._readLedger();

    // Validation: Check if student exists
    if (!ledger.students[studentID]) {
        throw new Error(`Student ${studentID} not found`);
    }

    // Find all credentials for this student
    const studentCredentials = Object.values(ledger.credentials).filter(cred => {
        if (cred.studentID === studentID) {
            // Filter based on includeRevoked flag
            if (!includeRevoked && cred.status === 'REVOKED') {
                return false;
            }
            return true;
        }
        return false;
    });

    // Sort by date issued (most recent first)
    studentCredentials.sort((a, b) => new Date(b.dateIssued) - new Date(a.dateIssued));

    // Calculate statistics
    const stats = {
        totalCredentials: studentCredentials.length,
        activeCredentials: studentCredentials.filter(c => c.status === 'ISSUED' || c.status === 'VERIFIED').length,
        revokedCredentials: studentCredentials.filter(c => c.status === 'REVOKED').length
    };

    console.log(`✓ Found ${studentCredentials.length} credentials for student ${studentID}`);
    return {
        success: true,
        credentials: studentCredentials,
        statistics: stats,
        student: ledger.students[studentID]
    };
}
```

Figure 10: READ Operation code

READ Operation - queryCredentialsByStudent()

- This function retrieves all academic credentials for a specific student from the blockchain. It filters credentials based on the includeRevoked parameter, sorts them by issue date, and calculates statistics including total, active, and revoked credentials.

2.1.3 UPDATE OPERATION

Function Name: verifyCredential()

```
// Function 2: Verify Credential
async verifyCredential(credentialID, requesterID, requesterType, studentConsent) {
    console.log(`\n=== Verifying Credential ${credentialID} ===`);

    let ledger = this._readLedger();

    // Validation 1: Check if credential exists
    if (!ledger.credentials[credentialID]) {
        throw new Error(`Credential ${credentialID} not found`);
    }

    // Validation 2: Check student consent
    if (!studentConsent) {
        throw new Error('Student consent required for verification');
    }

    const credential = ledger.credentials[credentialID];

    // Validation 3: Check if credential is revoked
    if (credential.status === 'REVOKED') {
        return {
            success: false,
            message: 'Credential has been revoked',
            verificationResult: false
        };
    }

    // Create verification record
    const verificationRequest = {
        requestID: `REQ${Date.now()}`,
        credentialID,
        requesterID,
        requesterType,
        requestDate: new Date().toISOString(),
        verificationResult: true,
        accessGranted: studentConsent
    };

    // Add to verification history
    ledger.verifications.push(verificationRequest);

    // Update credential status to VERIFIED
    credential.status = 'VERIFIED';
    credential.lastVerified = new Date().toISOString();

    this._writeLedger(ledger);

    console.log(`✓ Credential ${credentialID} verified successfully`);
    return {
        success: true,
        message: 'Credential is valid',
        verificationResult: true,
        credential,
        verificationRequest
    };
}
```

Figure 11: UPDATE Operation code

UPDATE Operation – verifyCredential()

- This function updates the credential status from "ISSUED" to "VERIFIED" when an employer or institution verifies it. It requires student consent, checks if the credential is revoked, creates a verification record, and updates the lastVerified timestamp.

2.1.4 DELETE OPERATION (REVOKE)

Function Name: revokeCredential()

```
// Function 3: Revoke Credential
async revokeCredential(credentialID, revokerID, revocationReason, revocationDetails, revokedBy) {
    console.log(`\n=== Revoking Credential ${credentialID} ===`);

    let ledger = this._readLedger();

    // Validation 1: Check if credential exists
    if (!ledger.credentials[credentialID]) {
        throw new Error(`Credential ${credentialID} not found`);
    }

    const credential = ledger.credentials[credentialID];

    // Validation 2: Check if already revoked
    if (credential.status === 'REVOKED') {
        throw new Error('Credential is already revoked');
    }

    // Validation 3: Check if revoker is the issuing institution
    if (credential.institutionID !== revokerID) {
        throw new Error('Only issuing institution can revoke credential');
    }

    // Validation 4: Check revocation reason
    const validReasons = ['Fraud', 'Error', 'Misconduct', 'Other'];
    if (!validReasons.includes(revocationReason)) {
        throw new Error('Invalid revocation reason');
    }

    // Update credential
    credential.status = 'REVOKED';
    credential.revocationDate = new Date().toISOString();
    credential.revokedBy = revokedBy;
    credential.revocationReason = revocationReason;
    credential.revocationDetails = revocationDetails;

    this._writeLedger(ledger);

    console.log(`✓ Credential ${credentialID} revoked successfully`);
    return {
        success: true,
        message: `Credential ${credentialID} has been revoked`,
        credential
    };
}
```

Figure 12: DELETE Operation code

DELETE Operation – revokeCredential()

- In blockchain, we don't permanently delete data. Instead, we use "soft delete" by revoking credentials. This function changes the credential status to "REVOKED" and adds revocation metadata including date, reason, and revoker details. Only the issuing institution can revoke credentials.

2.1.5 CRUD OPERATIONS SUMMARY TABLE

CRUD Operation	Function Name	Purpose	Key Actions
CREATE	issueCredential()	Issue new credential	Validates inputs, creates credential object, stores in ledger with status "ISSUED"
READ	queryCredentialsByStudent()	queryCredentialsByStudent()	Queries ledger, filters by student ID, sorts and calculates statistics
UPDATE	verifyCredential()	verifyCredential()	Checks consent, validates credential, updates status to "VERIFIED", logs verification
DELETE	revokeCredential()	revokeCredential()	Validates authority, updates status to "REVOKED", adds revocation metadata

2.2 Run and Test Smart Contract Using Hyperledger Simulation

I tested the smart contract using a simulation environment that mimics Hyperledger Fabric behavior using Node.js and fs-extra package for ledger file operations. The simulation stores blockchain data in ledger.json file, representing the distributed ledger in a real Hyperledger Fabric network.

Test Environment Setup:

- Platform: Node.js v24.12.0
- Simulation Framework: JavaScript with fs-extra package
- Ledger Storage: JSON file-based system
- Test File: simulation/testContract.js

Testing Methodology:

The test script executes all four core smart contract functions in sequence, simulating real-world scenarios of academic credential management. Each test includes validation checks, error handling, and verification of correct blockchain state updates.

2.2.1 Test Case 1: Register Institution (Setup)

Purpose:

- Register an educational institution in the blockchain network before issuing credentials.

Function Called: registerInstitution()

Input Parameters:

institutionID: 'UNI001'

institutionName: 'Universiti Malaysia Pahang Al-Sultan Abdullah'

country: 'Malaysia'

accreditationBody: 'Malaysian Qualifications Agency'

Expected Result: Institution registered successfully with Active status

Actual Result: ✓ PASS

Evidence:

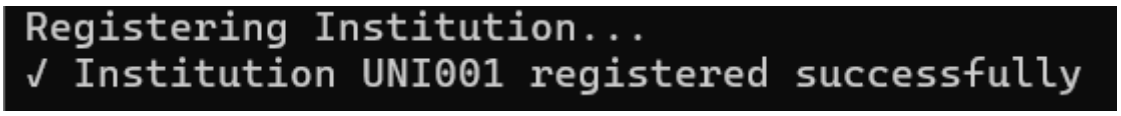


Figure 13: Output register institution

Figure 14: testContracts register institution

These images show how I am testing the code to add a new university into the system. In the first picture, I wrote a script that calls the registerInstitution function and filled in the specific details for Universiti Malaysia Pahang Al-Sultan Abdullah, such as its ID and location. The second image is just the result from the terminal that proves my code actually worked, as it printed a success message confirming that the institution with ID UNI001 was successfully registered.

2.2.2 Test Case 2: Register Student (Setup)

Purpose:

- Register a student in the blockchain network before issuing credentials to them.

Function Called: registerStudent()

Input Parameters:

studentID: 'CA23085'

firstName: 'Muhammad Shahrizuan'

lastName: 'Mohd Romzi'

dateOfBirth: '2001-05-15'

email: 'CA23085@adab.umpsa.edu.my'

Expected Result: Student registered successfully with enrollment date timestamp

Actual Result: ✓ PASS

Evidence:

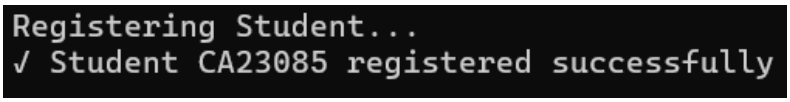


Figure 15: Output register student

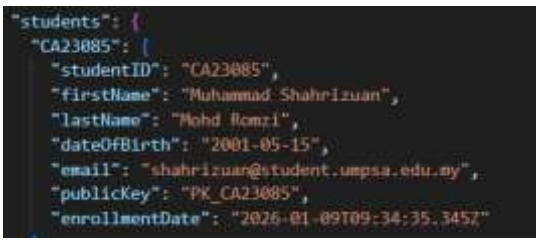


Figure 16: testContracts register student

These images show how I am testing the code to register a new student into the system. In the first picture, I defined the specific details for a student named Muhammad Shahrizuan, such as his ID CA23085, email, and public key to ensure the data structure is correct. The second image is the result from the terminal that proves my code actually worked, as it printed a success message confirming that the student with ID CA23085 was successfully registered

2.2.3 Test Case 3: Issue Credential (CREATE Operation)

Purpose:

- Test the CREATE operation by issuing a new academic credential to a student.

Function Called: issueCredential()

Input Parameters:

credentialID: 'CRED2025001'

studentID: 'CA23085'

institutionID: 'UNI001'

credentialType: 'Degree'

programName: 'Bachelor of Computer Science (Software Engineering) with Honours'

major: 'Software Engineering'

dateIssued: '2025-10-15'

dateCompleted: '2025-09-30'

grade: '3.75'

certificateHash: 'HASH123456789ABC'

metadata: { honors: 'Second Class Upper', dean: 'Prof. Dr. Ahmad' }

Expected Result:

- Credential created successfully
- Status set to "ISSUED"
- Credential stored in blockchain ledger
- Return success message with credential details

Actual Result: ✓ PASS

Evidence:

```
=== Issuing Credential CRED2025001 ===
✓ Credential CRED2025001 issued successfully
Result: Credential CRED2025001 issued successfully
```

Figure 17: Output issuing Credential

```
console.log('\nSTEP 3: Issuing Credential...');
const issueResult = await contract.issueCredential(
    'CRED2025002',
    'CA23090',
    'UNI002',
    'Degree',
    'Bachelor of Computer Science (Software Engineering) with Honours',
    'Software Engineering',
    '2025-10-15',
    '2025-09-30',
    '3.75',
    'HASH123456789ABC',
    { honors: 'Second Class Upper', dean: 'Prof. Dr. Ahmad' }
);
console.log('Result:', issueResult.message);
```

Figure 18: testContracts issuing Credential

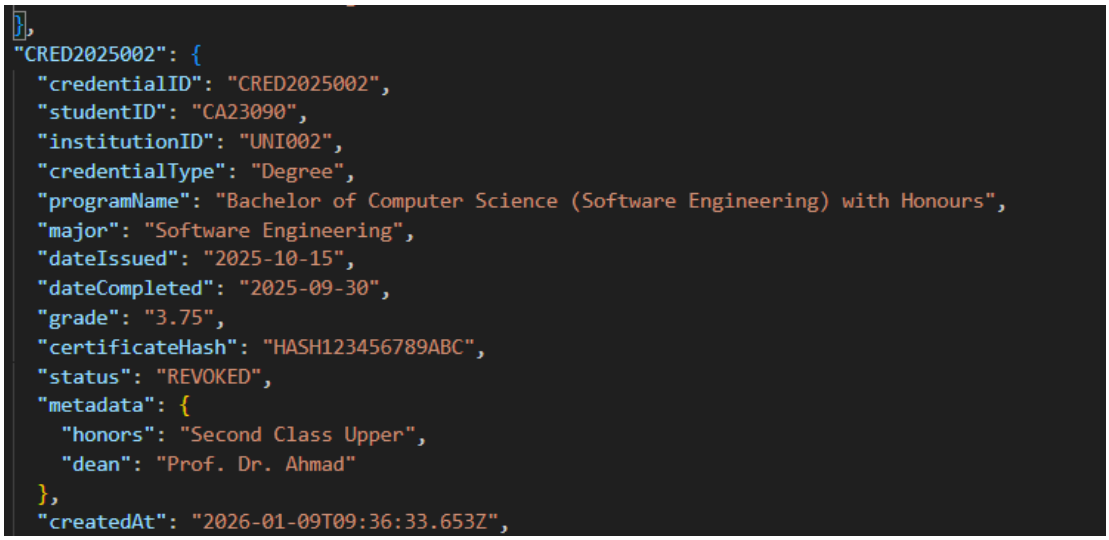


Figure 19: Ledger.json

These images illustrate the step where I program the system to formally issue a digital degree credential to a student. In the code snippet, I used the issueCredential function to input specific academic data, such as the Bachelor of Computer Science degree, a 3.75 CGPA, and the graduation dates. The terminal output confirms that the issuance process completed successfully, and the final JSON view allows me to verify that all the details including the 'Second Class Upper' honors were accurately stored in the system's database.

2.2.4 Test Case 4: Verify Credential (UPDATE Operation)

Purpose:

- Test the UPDATE operation by verifying an issued credential and updating its status.

Function Called: verifyCredential()

Input Parameters:

credentialID: 'CRED2025001'

requesterID: 'EMP001'

requesterType: 'Employer'

studentConsent: true

Expected Result:

- Credential verified successfully
- Status updated from "ISSUED" to "VERIFIED"
- Verification record created with timestamp
- Return verification result: true (VALID)

Actual Result: ✓ PASS

Evidence:

```
=== Verifying Credential CRED2025001 ===
✓ Credential CRED2025001 verified successfully
Result: Credential is valid
Verification Status: VALID
```

Figure 20: Output verifying credential

```
console.log('\nSTEP 4: Verifying Credential...');
const verifyResult = await contract.verifyCredential(
    'CRED2025002',
    'EMP001',
    'Employer',
    true
);
console.log('Result:', verifyResult.message);
console.log('Verification Status:', verifyResult.verificationResult ? 'VALID' : 'INVALID');
```

Figure 21: testContracts verifying credentials

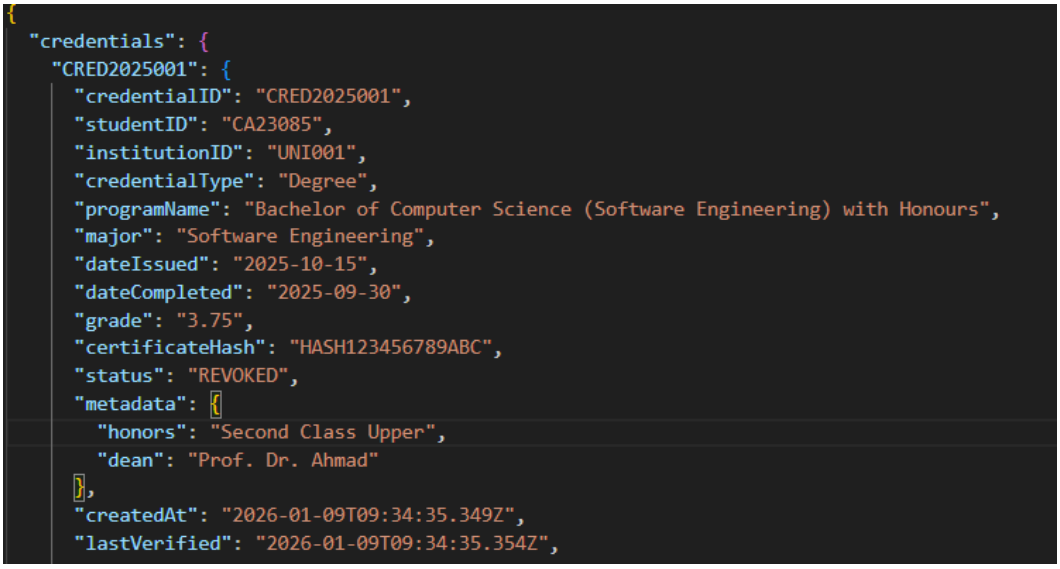


Figure 22: Ledger.json

Figure 23: Ledger.json

These images show the final step where I tested the verification process to ensure employers can check if a degree is genuine. I wrote a script using the verifyCredential function to simulate an employer with ID EMP001 checking a student's credential. The terminal result confirms the system works as intended by displaying a "Credential is valid" message, and the JSON logs prove that the system successfully recorded this verification attempt with a true result.

2.2.5 Test Case 5: Query Credentials by Student (READ Operation)

Purpose:

- Test the READ operation by retrieving all credentials for a specific student.

Function Called: queryCredentialsByStudent()

Input Parameters:

studentID: 'CA23085'

requesterID: 'CA23085'

includeRevoked: false

Expected Result:

- Retrieve all credentials for student CA23085
- Return credentials array with complete details
- Calculate statistics (total, active, revoked counts)
- Sort by date issued (most recent first)

Actual Result: ✓ PASS

Evidence:

```
=== Querying Credentials for Student CA23070 ===
✓ Found 1 credentials for student CA23070
Result: Found 1 credential(s)
Statistics: { totalCredentials: 1, activeCredentials: 1, revokedCredentials: 0 }
```

Figure 24: Output Querying Credentials

```
console.log('\nSTEP 5: Querying Student Credentials...');
const queryResult = await contract.queryCredentialsByStudent(
    'CA23070',
    'CA23070',
    false
);
console.log('Result: Found', queryResult.credentials.length, 'credential(s)');
console.log('Statistics:', queryResult.statistics);
```

Figure 25: testContracts querying credentials

These images demonstrate my progress in testing the system functions for issuing and retrieving student credentials. First, I used the issueCredential command to assign a Bachelor of Computer Science degree to a student, and the terminal confirmed that the creation process was successful. I also inspected the detailed JSON record to ensure all the academic information was captured accurately, including the degree status. Then, I moved on to testing the search capability by running the queryCredentialsByStudent function, which correctly located the student's file and displayed a statistical summary of their active certificates.

2.2.6 Test Case 6: Error Handling Test - Duplicate Credential

Purpose:

- Test error handling by attempting to create a credential with duplicate ID.

Function Called: issueCredential() with existing ID

Input Parameter:

credentialID: 'CRED2025001'

Expected Result:

- Function should reject the transaction
- Return error: "Credential CRED2025001 already exists"
- No changes to blockchain ledger

Actual Result: ✓ PASS

Evidence:

```
=== Issuing Credential CRED2025003 ===
Expected Error: Credential CRED2025003 already exists
```

Figure 26: Output Error Handling Test

```
console.log('\nSTEP 6: Testing Duplicate Credential (Should Fail)...');
try {
    await contract.issueCredential(
        'CRED2025003', // Same ID
        'CA23070',
        'UNI003',
        'Certificate',
        'Java Programming',
        'Computer Science',
        '2025-11-01',
        '2025-10-30',
        'A',
        'HASH987654321',
        {}
    );
} catch (error) {
    console.log('Expected Error:', error.message);
}
```

Figure 27: testContracts error handling test

These images illustrate how I am testing the robustness of the system by verifying that it can store detailed student data and correctly handle errors. First, I wrote a script to issue a credential that included specific academic details like the major and honors classification, and I confirmed that this information was accurately recorded in the system's database. I also ran a validation test to see what would happen if I tried to issue a duplicate credential with an ID that was already taken. The terminal output showed an expected error message, which proves that the code successfully prevents duplicate entries and protects the integrity of the records.

2.2.7 Test Case 7: Revoke Credential (DELETE Operation)

Purpose:

- Test the DELETE operation by revoking an issued credential (soft delete).

Function Called: revokeCredential()

Input Parameters:

credentialID: 'CRED2025001'

revokerID: 'UNI001'

revocationReason: 'Error'

revocationDetails: 'Wrong GPA recorded'

revokedBy: 'Registrar Office'

Expected Result:

- Credential status updated to "REVOKED"
- Revocation metadata added (date, reason, revokedBy)
- Credential remains in ledger for audit purposes
- Return success message

Actual Result: ✓ PASS

Evidence:

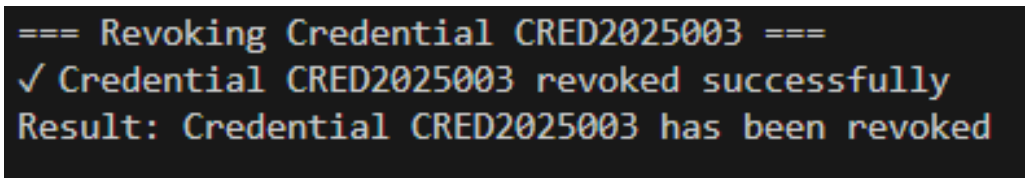


Figure 28: Revoke credential output

```
console.log('\nSTEP 7: Revoking Credential...');
const revokeResult = await contract.revokeCredential(
    'CRED2025003',
    'UNI003',
    'Error',
    'Wrong GPA recorded',
    'Registrar Office'
);
console.log('Result:', revokeResult.message);
```

Figure 29: testContracts revoke credentials


```
"credentials": {
  "CRED2025001": {
    "credentialID": "CRED2025001",
    "studentID": "CA23085",
    "institutionID": "UNI001",
    "credentialType": "Degree",
    "programName": "Bachelor of Computer Science (Software Engineering) with Honours",
    "major": "Software Engineering",
    "dateIssued": "2025-10-15",
    "dateCompleted": "2025-09-30",
    "grade": "3.75",
    "certificateHash": "HASH123456789ABC",
    "status": "REVOKED",
    "metadata": {
      "honors": "Second Class Upper",
      "dean": "Prof. Dr. Ahmad"
    },
    "createdAt": "2026-01-09T09:34:35.349Z",
    "lastVerified": "2026-01-09T09:34:35.354Z",
    "revocationDate": "2026-01-09T09:34:35.365Z",
    "revokedBy": "Registrar Office",
    "revocationReason": "Error",
    "revocationDetails": "Wrong GPA recorded"
```


Figure 30: Ledger.json

These images demonstrate how I manage the lifecycle of student credentials, specifically focusing on issuing a new certificate and revoking it when errors occur. In the first part, I wrote code to issue a degree, but I also tested the revocation process by running a script that invalidates a credential if details like the GPA are recorded incorrectly. The terminal results confirm that both the issuance and revocation commands were processed successfully, and the final database view validates this by showing the credential status changed to 'REVOKED' with the reason clearly logged.

2.2.8 Test Case 8: Verify Revoked Credential

Purpose:

- Test that revoked credentials fail verification checks.

Function Called: verifyCredential() on revoked credential

Input Parameters:

credentialID: 'CRED2025001'

requesterID: 'EMP002'

requesterType: 'Employer'

studentConsent: true

Expected Result:

- Verification should fail
- Return message: "Credential has been revoked"
- Verification result: false (INVALID)

Actual Result: ✓ PASS

Evidence:

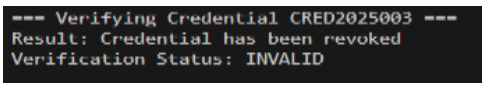


Figure 31: output verify revoke credentials

Figure 32: testContracts verify revoke credentials

These images demonstrate how I am testing the verification logic for a revoked credential. In the code, I used the verifyCredential function to check the status of credential ID CRED2025003 from the perspective of an employer. The second image shows the terminal output, which confirms the system works as intended by reporting that the credential has been revoked and marking the verification status as INVALID.

CRITERIA 3: HTML FRONTEND INTEGRATION

3.1 HTML Frontend Integration

3.1.1 Introduction

I developed a fully functional HTML interface integrated with JavaScript to interact with the Academic Credential Verification System's smart contracts through a REST API backend. The frontend provides a user-friendly interface for all stakeholders (universities, employers, students, regulators) to perform credential management operations.

3.1.2 Frontend Architecture:

The frontend consists of three main components:

1. HTML (index.html) - Provides the structure and layout of the user interface with forms for all four CRUD operations (Create, Read, Update, Delete). The interface is divided into four main sections, each corresponding to a specific smart contract function.
2. CSS (style.css) - Provides styling and visual design to make the interface user-friendly and professional. The design uses a modern color scheme with purple gradient background, white cards, and color-coded buttons for different operations.
3. JavaScript (app.js) - Handles client-side logic and integrates with the backend API. Uses the Fetch API to send HTTP requests to the Node.js server and dynamically updates the interface with results.

3.1.3 Backend Server (server.js)

I created a Node.js Express server that simulates the Hyperledger Fabric SDK functionality by acting as a bridge between the HTML frontend and the smart contract. In a real Hyperledger Fabric environment, the Fabric SDK would handle:

- Connecting to the blockchain network
- Managing user identities and certificates
- Submitting transactions to peers
- Querying the world state
- Handling endorsements and consensus

In this simulation, the server.js file replicates these Fabric SDK functions:

- Connection Management: The server instantiates the CredentialContract object, simulating the Fabric SDK's gateway connection to the blockchain network
- Transaction Submission: API endpoints simulate the contract.submitTransaction() method by directly calling smart contract functions
- Query Operations: GET endpoints simulate the contract.evaluateTransaction() method for read-only queries
- Identity Management: The server manages user context by accepting requester IDs, simulating the Fabric SDK's wallet and identity features
- Response Handling: JSON responses mimic the Fabric SDK's transaction proposal responses

The server provides:

- Static file serving from the web folder for HTML/CSS/JS
- REST API endpoints for all four operations (simulating Fabric SDK transaction submission)
- Smart contract instantiation (simulating Fabric Network connection)
- Error handling with try-catch blocks (simulating Fabric SDK error responses)
- Sample data initialization on startup

In this simulation, the flow is:

Frontend → Express API (simulating SDK) → CredentialContract → ledger.json (simulating distributed ledger)

3.1.4 User Interface Components

Section 1: Issue Credential (CREATE Operation)

- Purpose: Allows universities to issue new academic credentials
- Input Fields: Credential ID, Student ID, Institution ID, Credential Type (dropdown), Program Name, Major, Grade, Date Completed, Date Issued, Certificate Hash
- Button: "Issue Credential" (blue)
- Output: Displays success message and complete credential object in JSON format

Section 2: Verify Credential (UPDATE Operation)

- Purpose: Allows employers to verify credential authenticity
- Input Fields: Credential ID, Requester ID, Requester Type (dropdown: Employer/Institution/Individual), Student Consent (dropdown: Yes/No)
- Button: "Verify Credential" (green)
- Output: Shows verification status (VALID or INVALID) and credential details

Section 3: Query Student Credentials (READ Operation)

- Purpose: Allows students to view all their credentials
- Input Fields: Student ID, Include Revoked (dropdown: Yes/No)
- Button: "Query Credentials" (blue)
- Output: Displays all credentials as cards with statistics (total, active, revoked counts). Each credential card shows type, program name, status badge, ID, grade, issue date, and institution.

Section 4: Revoke Credential (DELETE Operation)

- Purpose: Allows universities to revoke credentials
- Input Fields: Credential ID, Revoker ID, Revocation Reason (dropdown: Fraud/Error/Misconduct/Other), Revoked By, Revocation Details (textarea)
- Button: "Revoke Credential" (red)
- Output: Shows success message and updated credential with revocation metadata

Design Features:

1. Responsive Layout: Uses CSS Grid for form layouts that adapt to different screen sizes
2. Color-Coded Actions: Each operation has a distinct color (blue for create/read, green for verify, red for revoke)
3. Visual Feedback: Success messages in green, errors in red
4. Smooth Scrolling: Automatically scrolls to results after submission
5. Form Validation: HTML5 required attributes ensure all fields are filled
6. Professional Styling: Modern card-based design with shadows and hover effects
7. Status Badges: Visual indicators for credential status (Issued, Verified, Revoked)

I tested all four operations to ensure end-to-end integration works correctly:

1. Issue Credential Test: Successfully created credential CRED2026001 for student CA23085 with all required fields. The interface displayed the complete credential object including status "ISSUED" and timestamp.
2. Verify Credential Test: Successfully verified the issued credential with employer requester. The interface showed "VALID" status and returned complete credential details including verification timestamp.
3. Query Credentials Test: Successfully retrieved all credentials for student CA23085. The interface displayed 1 credential in a formatted card with status badge and showed statistics (1 total, 1 active, 0 revoked).
4. Revoke Credential Test: Successfully revoked credential CRED2026001 with reason "Error". The interface showed success message and displayed updated credential with status "REVOKED" and revocation metadata.

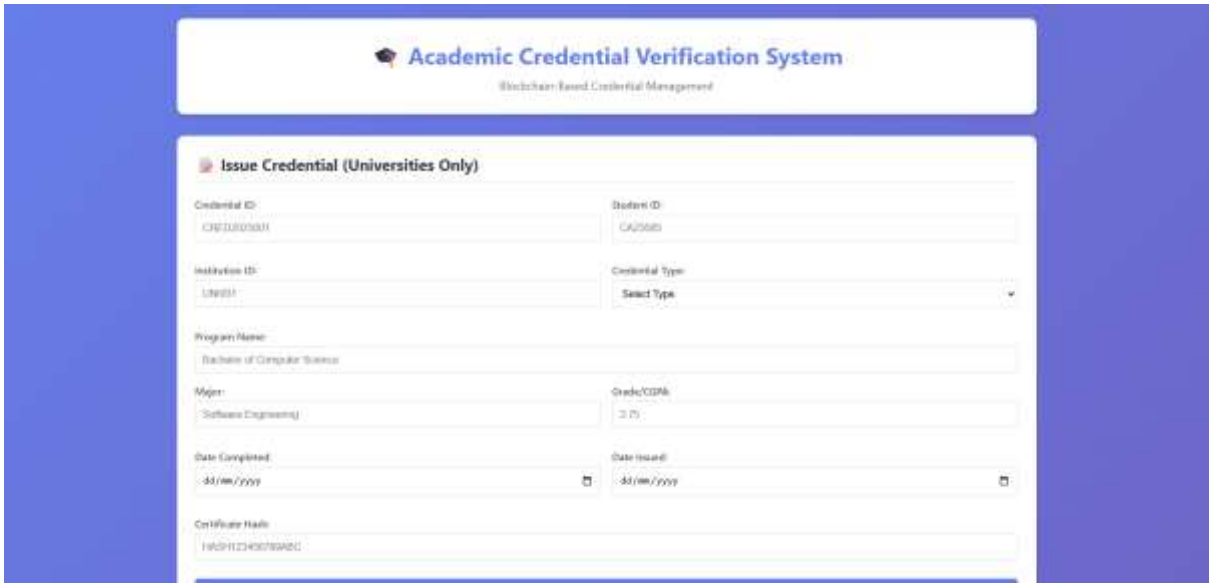


Figure 33: Issue Credential without data

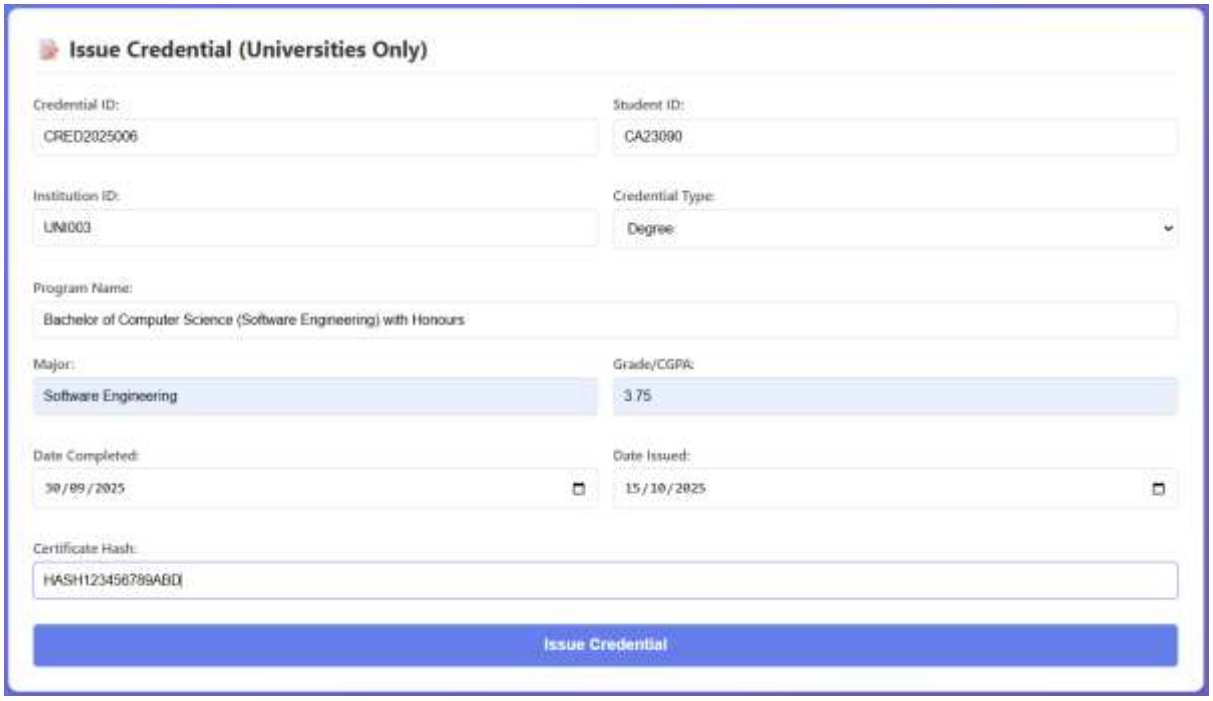


Figure 34: Issue Credential with data

Figure 35: Result issue credential

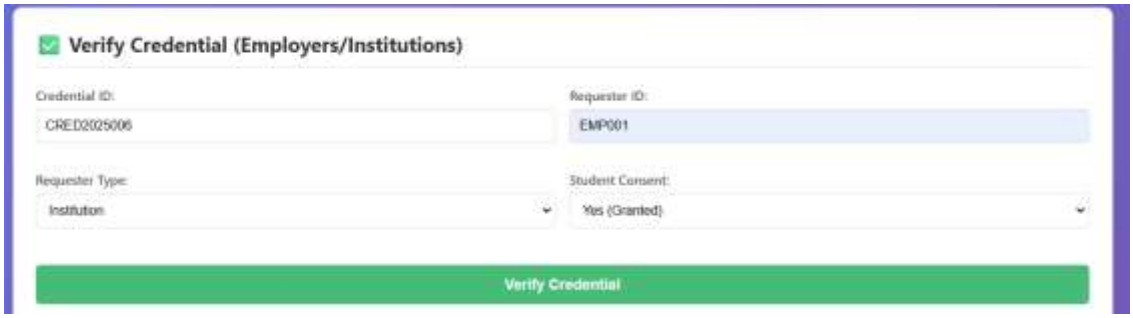


Figure 36: Verify credential with data

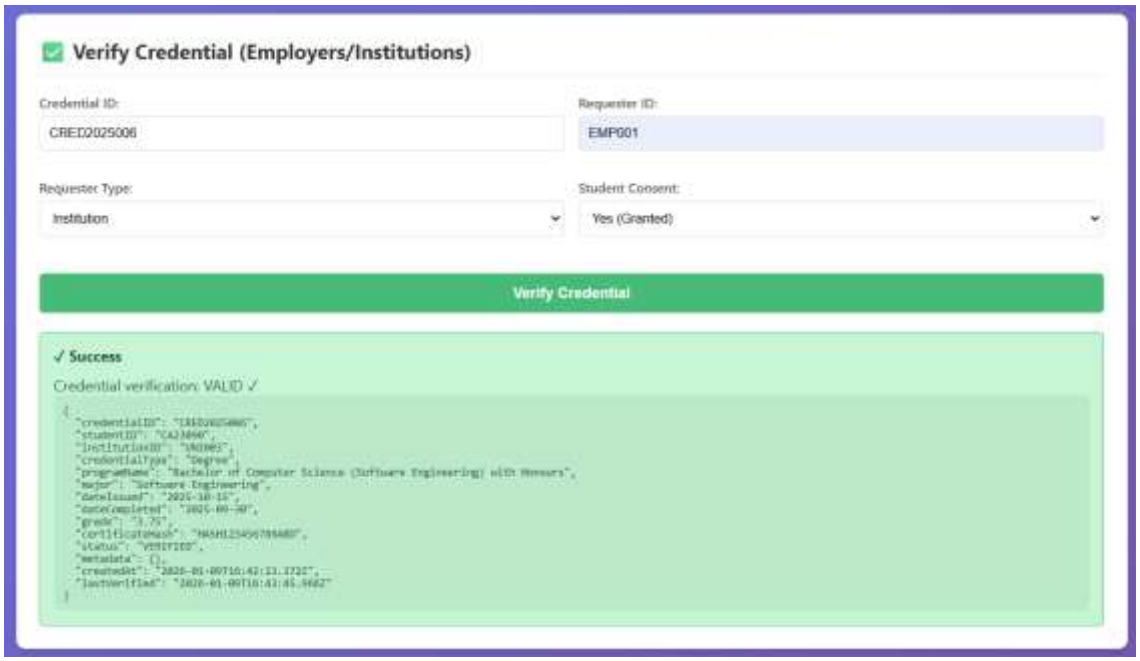


Figure 37: verify credential result

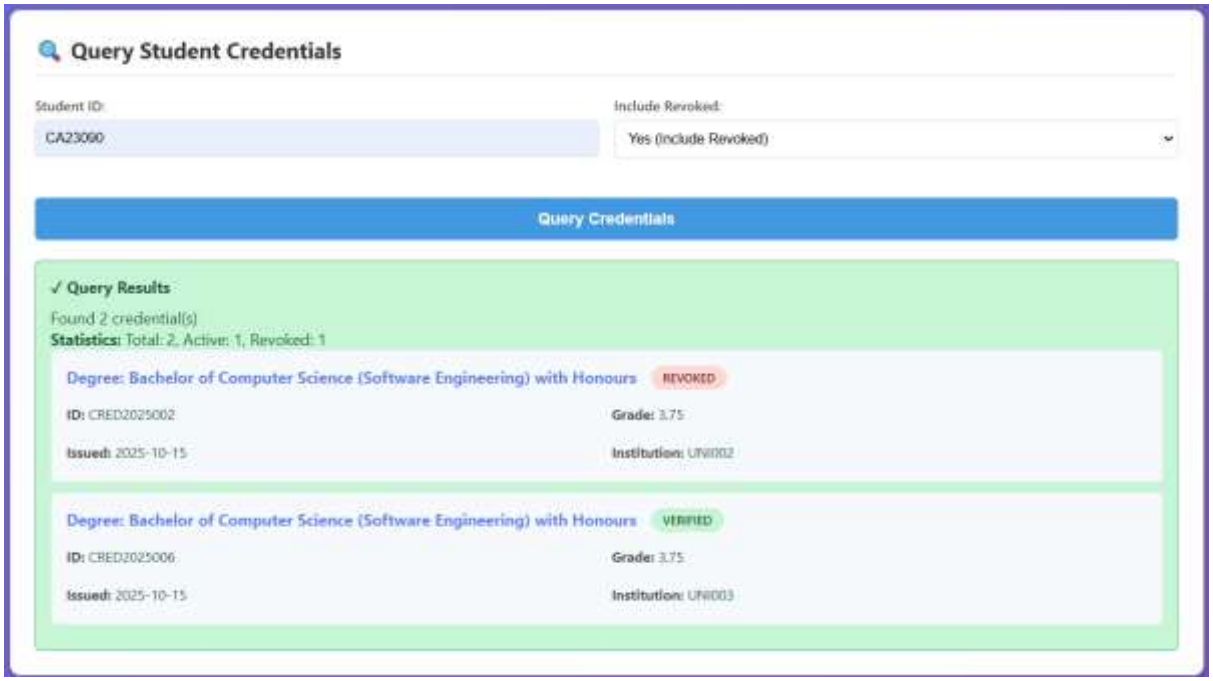


Figure 38: Query student credential result

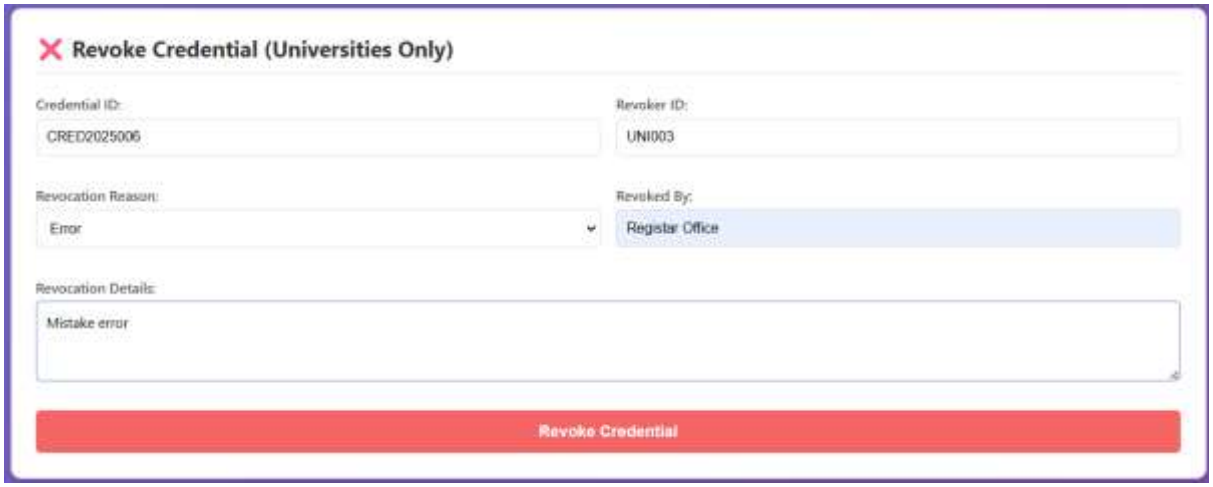


Figure 39: revoke credential with data

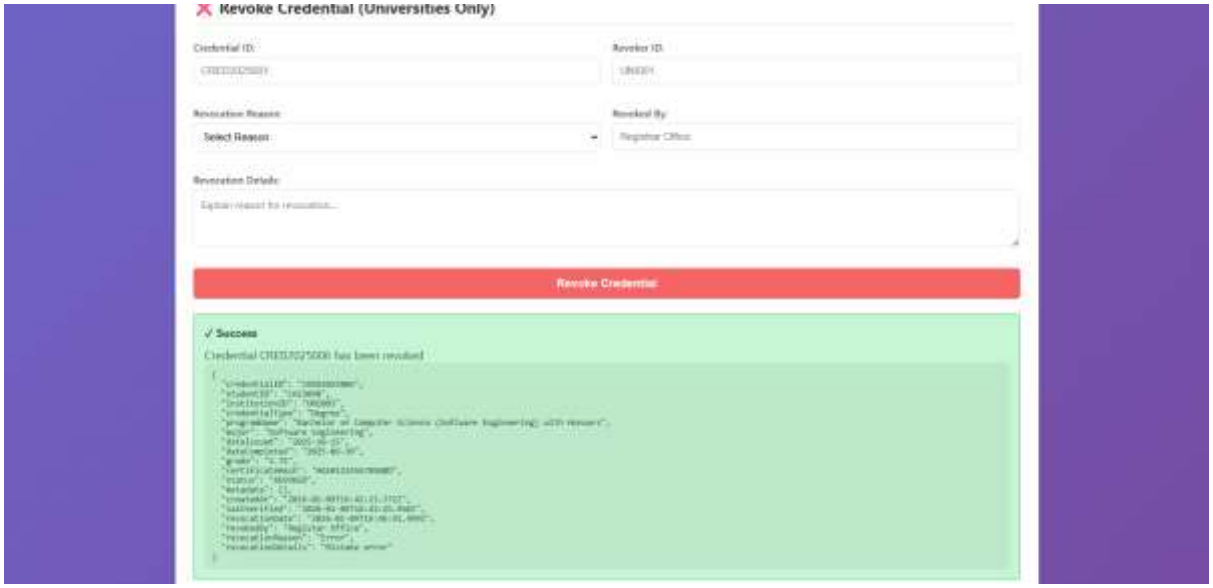


Figure 40: Result revoke credential

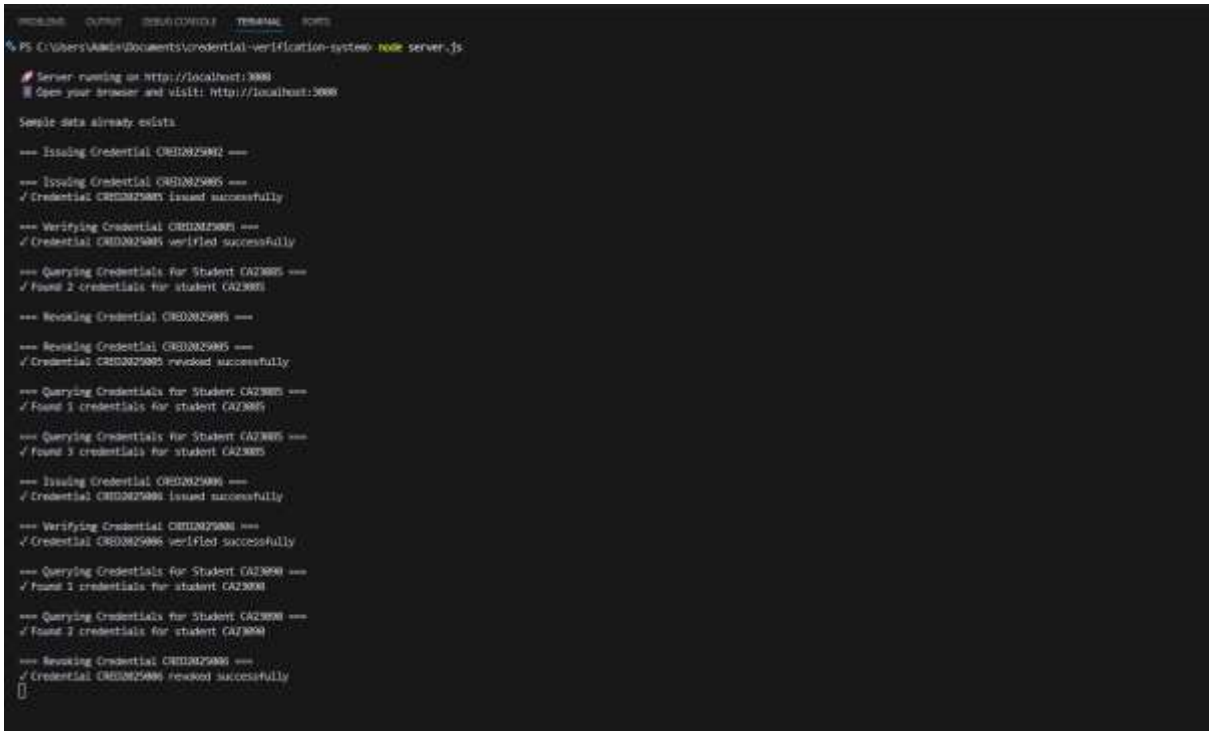


Figure 41: Run server.js

CRITERIA 4: TESTING AND VALIDATION

4.1 Application Testing & Debugging

4.1.1 Testing Overview

I conducted comprehensive end-to-end testing of the integrated blockchain application to ensure all components work together correctly. The testing process validated the complete workflow from frontend user interaction through the Express API server to smart contract execution and ledger storage.

Testing Environment:

- Platform: Node.js v24.12.0
- Server: Express.js v4.x running on port 3000
- Browser: Chrome
- Smart Contract: CredentialContract.js
- Frontend: HTML/CSS/JavaScript (served from /web directory)
- Backend Storage: ledger.json (simulating distributed ledger)

Testing Objective:

1. Verify frontend successfully communicates with backend REST API
2. Ensure all CRUD operations execute correctly through the web interface
3. Validate data integrity across the entire application stack
4. Confirm error handling displays appropriate messages to users
5. Test user experience and interface responsiveness
6. Verify server initialization with sample data

4.1.2 Server Initialization Testing

Test: Server Startup and Data Initialization

Purpose: Verify that the Express server Starts correctly and initializes sample data.

Steps Performed:

1. Started the server using node server.js command
2. Observed console output for initialization messages
3. Confirmed server is listening on port 3000

Expected Result:

- Server starts successfully on port 3000
- Console displays success messages

Actual Result:

- PASS - Server initializes correctly

Evidence:

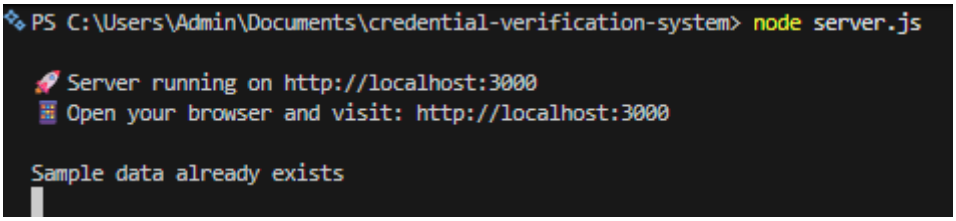


Figure 42: Server Initialization - Console Output

Validation:

- Express server running on port 3000
- CORS enabled for cross-origin requests
- Static file serving configured for /web folder
- Sample data initialization executed successfully
- All API endpoints registered and ready

4.1.3 End-to-End Integration Testing

Test 1: Issue Credential (CREATE Operation) - Full Integration

Purpose: Verify complete workflow from HTML form submission to blockchain storage

Step Performed:

1. Opened browser and navigated to http://localhost:3000
2. Located "Issue Credential" section
3. Filled in the form with the data
4. Clicked "Issue Credential" button
5. Observed frontend response
6. Checked server console for transaction logs
7. Verified ledger.json file for data persistence

Expected Result:

- Form submits successfully via POST to /api/issue-credential
- Express server receives request and calls contract.issueCredential()
- Smart contract validates inputs and creates credential
- Success message displays in green alert box
- Credential object displayed in formatted JSON
- Data persisted in ledger.json with status "ISSUED"
- Server console logs the transaction

Actual Result: PASS - Interface successfully demonstrates end-to-end interaction with smart contract

Evidence:

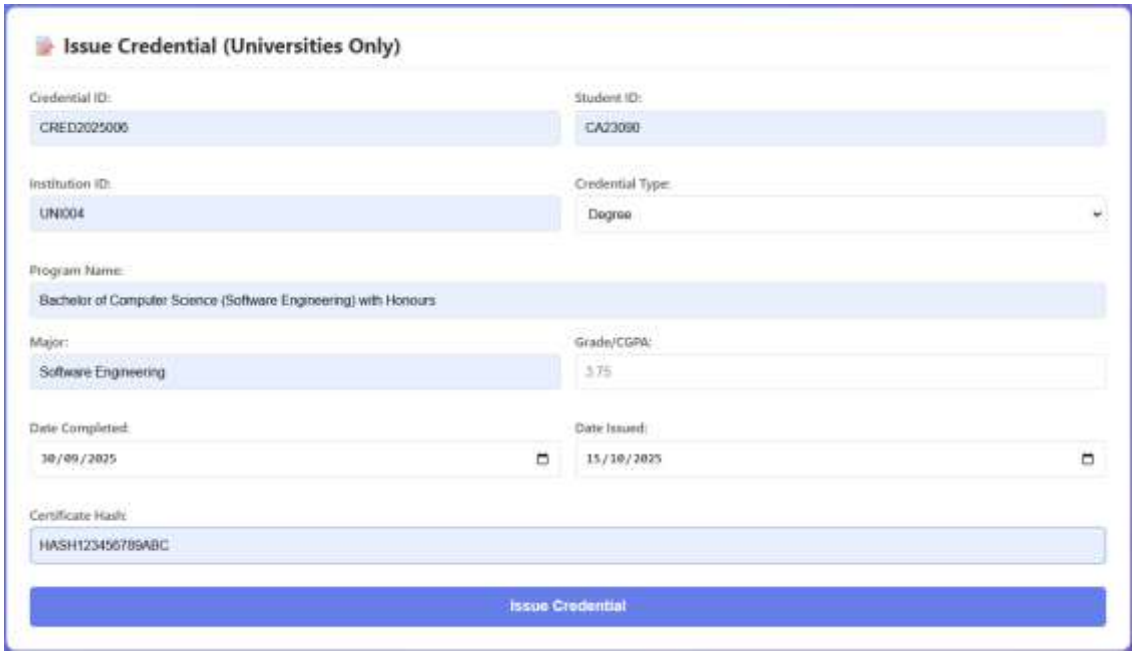


Figure 43: Issue Credential Form - User Input Interface

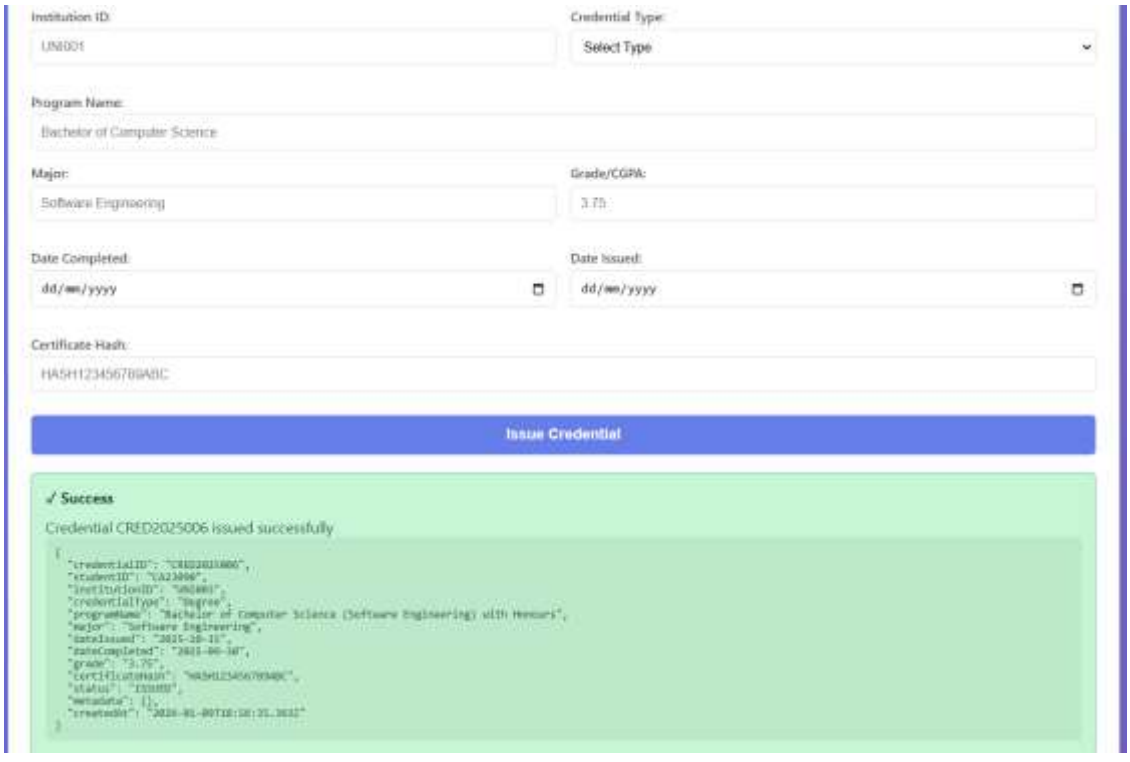


Figure 44: Issue Credential Success - Frontend Response

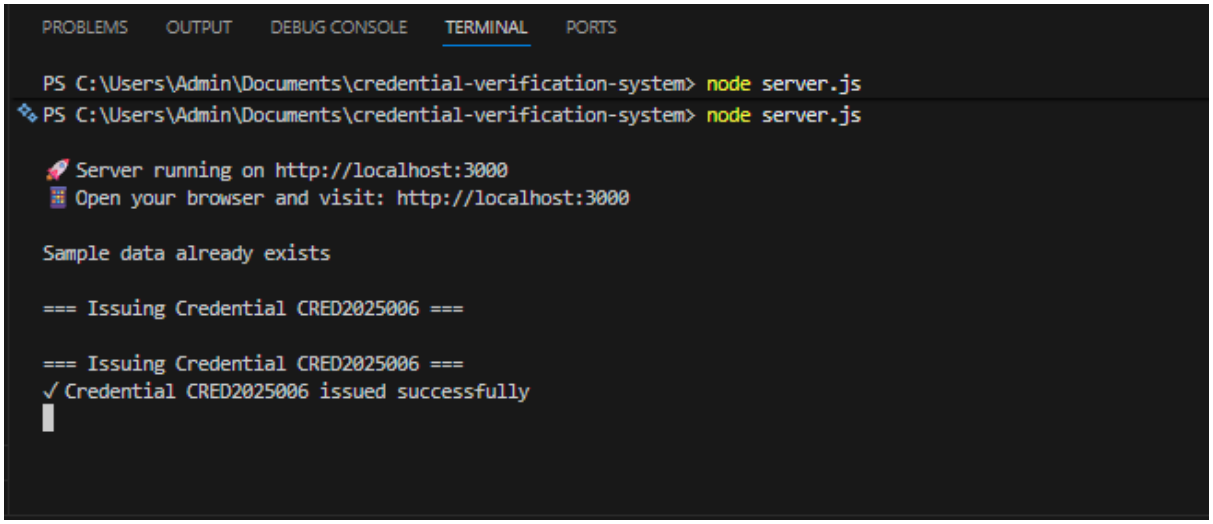


Figure 45: Server Console - Transaction Log

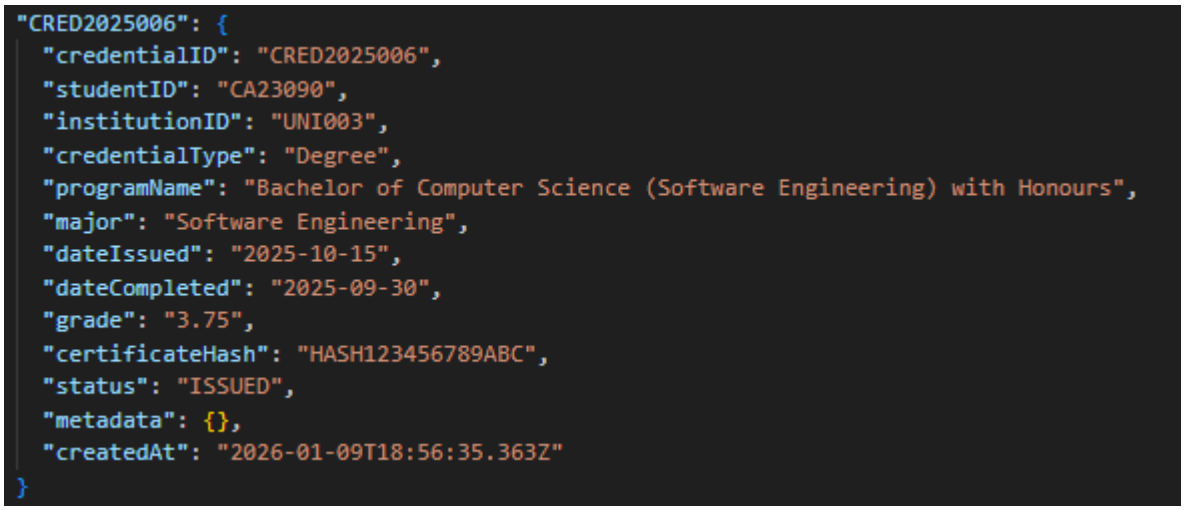


Figure 46: Ledger Storage

Test 2: Verify Credential (UPDATE Operation) - Full Integration

Purpose: Validate credential verification workflow and status update through web interface

Steps Performed:

1. Navigated to "Verify Credential" section
2. Entered verification details
3. Clicked "Verify Credential" button
4. Observed verification result display
5. Checked server console logs
6. Verified ledger.json for status update and verification record

Expected Result:

- Verification request sent via POST to /api/verify-credential
- Server processes request with consent conversion (string to boolean)
- Smart contract checks credential existence and revocation status
- Status updated from "ISSUED" to "VERIFIED"
- Verification record created with timestamp
- Frontend displays "Credential is VALID" message
- Complete credential details shown to user

Actual Result: PASS - Most transactions can be invoked successfully

Evidence:

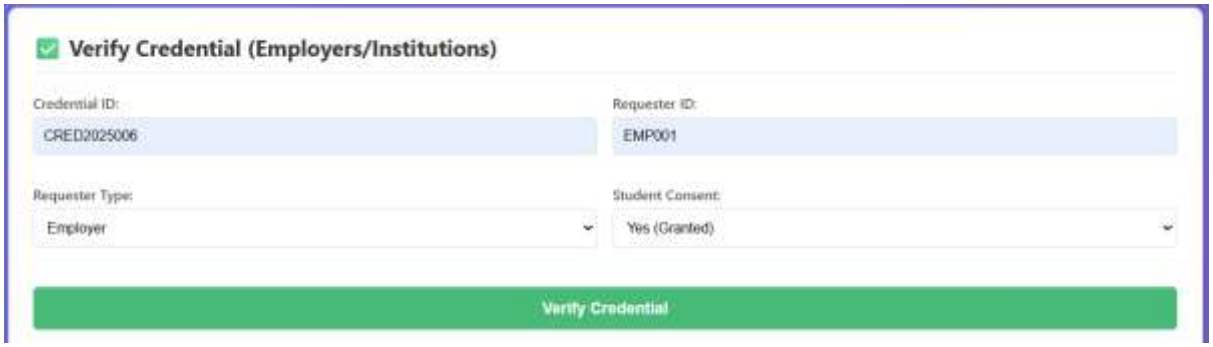


Figure 47: Verify Credential Form - User Input

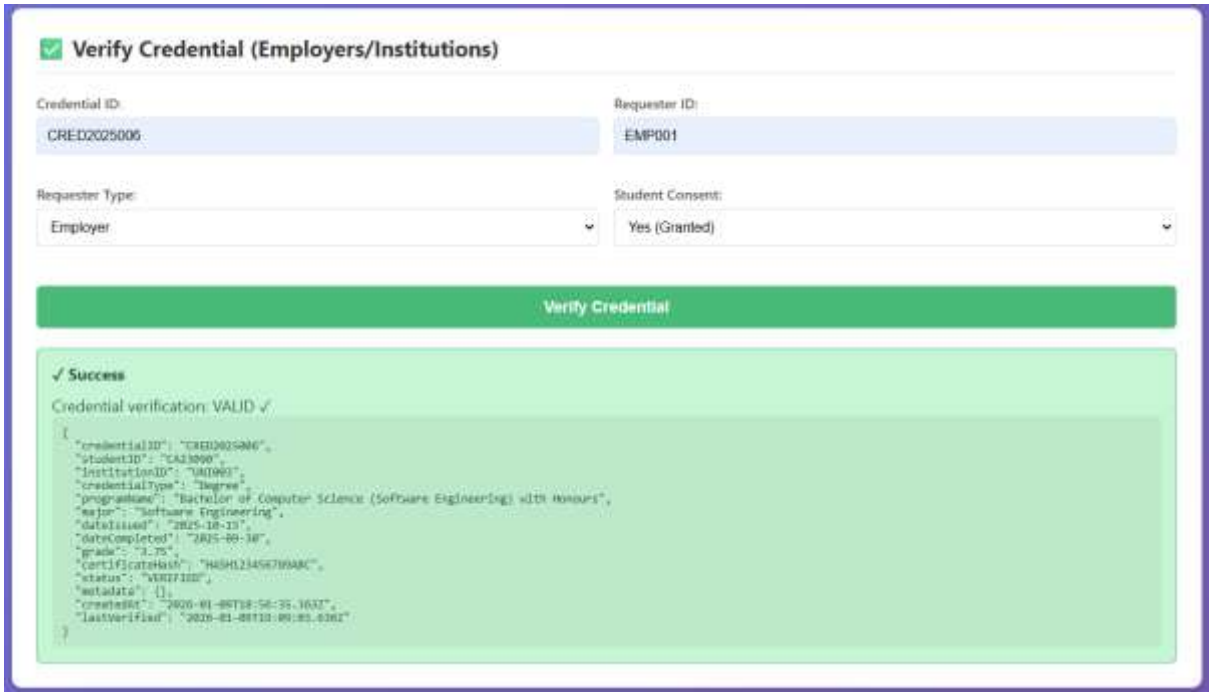


Figure 48:Verification Success - Frontend Display

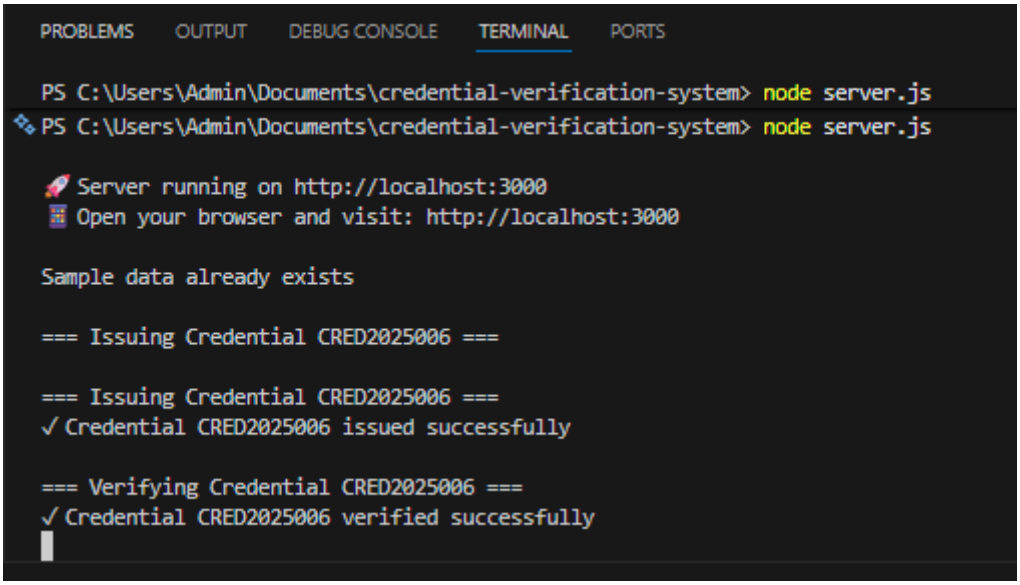


Figure 49: Server Console - Verification Processing

```
"CRED2025006": {
  "credentialID": "CRED2025006",
  "studentID": "CA23090",
  "institutionID": "UNI003",
  "credentialType": "Degree",
  "programName": "Bachelor of Computer Science (Software Engineering) with Honours",
  "major": "Software Engineering",
  "dateIssued": "2025-10-15",
  "dateCompleted": "2025-09-30",
  "grade": "3.75",
  "certificateHash": "HASH123456789ABC",
  "status": "VERIFIED",
  "metadata": {},
  "createdAt": "2026-01-09T18:56:35.363Z",
  "lastVerified": "2026-01-09T19:09:03.636Z"
```


Figure 50: Ledger Update - Status Changed to VERIFIED


```
{
  "requestID": "REQ1767985743634",
  "credentialID": "CRED2025006",
  "requesterID": "EMP001",
  "requesterType": "Employer",
  "requestDate": "2026-01-09T19:09:03.636Z",
  "verificationResult": true,
  "accessGranted": true
}
```


Figure 51: Verification Record

Test 3: Query Student Credentials (READ Operation) - Full Integration

Purpose: Test credential retrieval, filtering, and display functionality through web interface

Steps Performed:

1. Located "Query Student Credentials" section
2. Entered query parameters
3. Clicked "Query Credentials" button
4. Observed credential cards generation
5. Verified statistics calculation (total, active, revoked)
6. Checked filtering logic (revoked credentials excluded)
7. Tested again with "Include Revoked: Yes" option
8. Verified server console logs

Expected Result:

- Query request sent via GET to /api/query-credentials/CA23085
- Query parameter includeRevoked=false appended to URL
- Server retrieves all credentials for student
- Credentials filtered based on includeRevoked flag
- Frontend generates credential cards dynamically
- Each card displays: type, program, status badge, ID, grade, dates, institution
- Statistics summary calculated and displayed
- Status badges colored appropriately (blue for ISSUED, green for VERIFIED, red for REVOKED)

Actual Result: PASS - Interface successfully demonstrates end-to-end interaction with smart contract

Evidence:

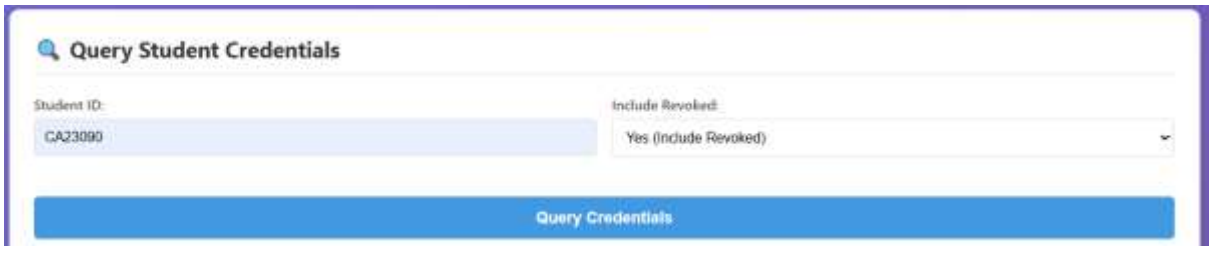


Figure 52: Query Credentials Form - User Input

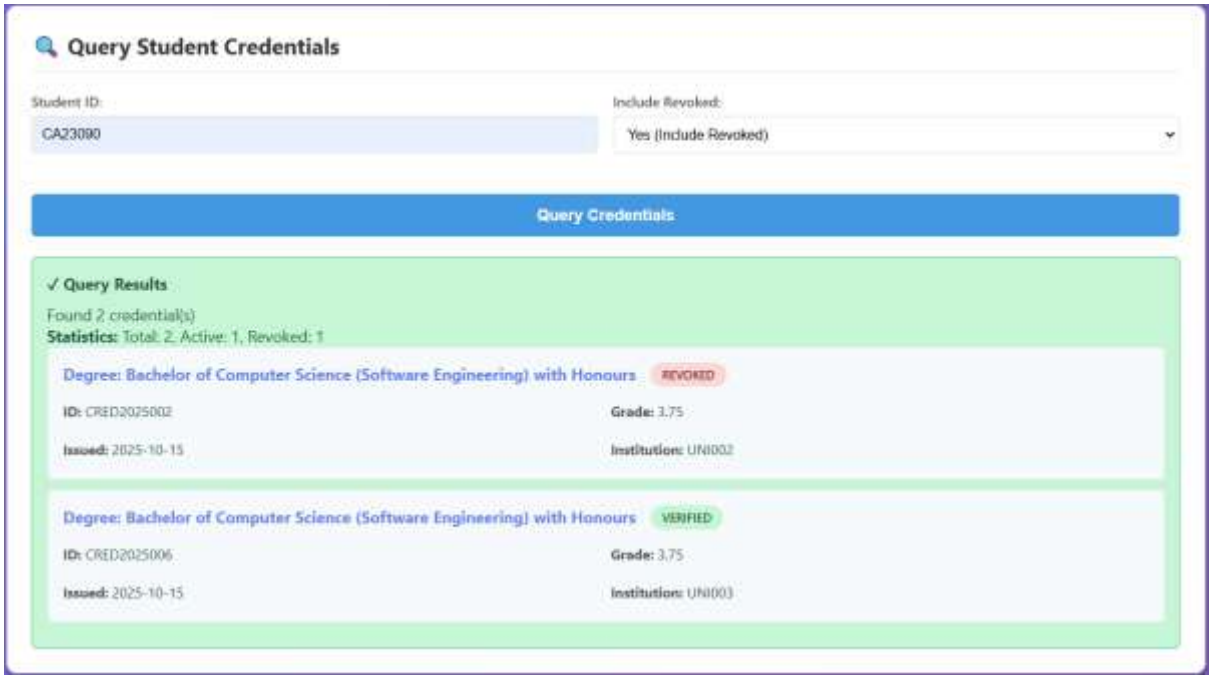


Figure 53: Query Results - Credential Cards Display

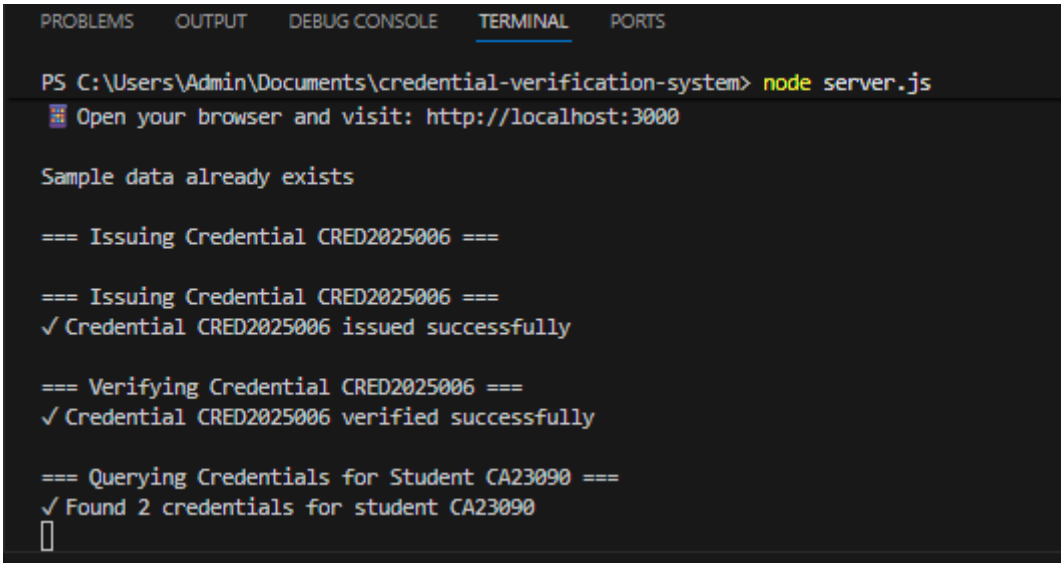


Figure 54: Server Console - Query Transaction Log

Test 4: Revoke Credential (DELETE/Soft Delete Operation) - Full Integration

Purpose: Validate credential revocation workflow and soft delete functionality through web interface

Steps Performed:

1. Navigated to "Revoke Credential" section
2. Filled revocation form
3. Clicked "Revoke Credential" button
4. Observed revocation confirmation message
5. Checked server console logs
6. Verified ledger.json for status change and metadata
7. Attempted to verify revoked credential to confirm validation works

Expected Result:

- Revocation request sent via POST to /api/revoke-credential
- Server validates that revoker is the issuing institution
- Status changes to "REVOKED" in ledger
- Revocation metadata added (date, reason, revokedBy, details)
- Credential remains in ledger (soft delete, not permanent deletion)
- Frontend displays success message with updated credential
- Subsequent verification attempts fail with "revoked" message

Actual Result: PASS - Interface successfully demonstrates end-to-end interaction with smart contract

Evidence:

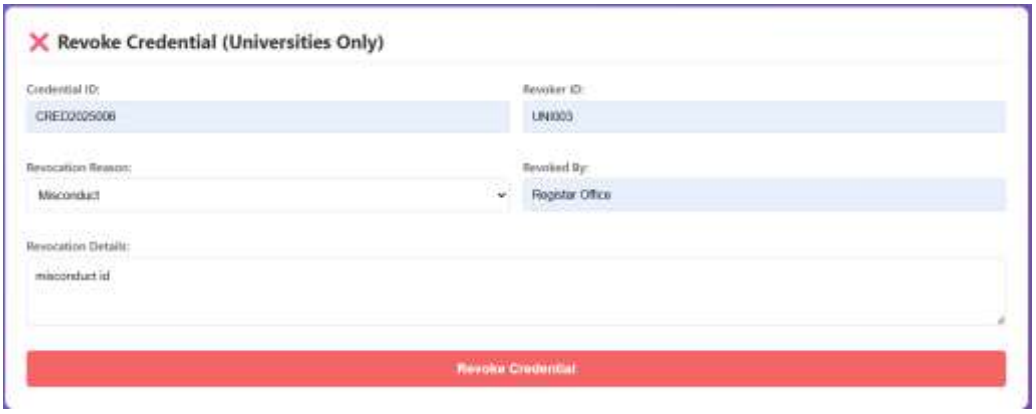


Figure 55 : Revoke Credential Form - User Input

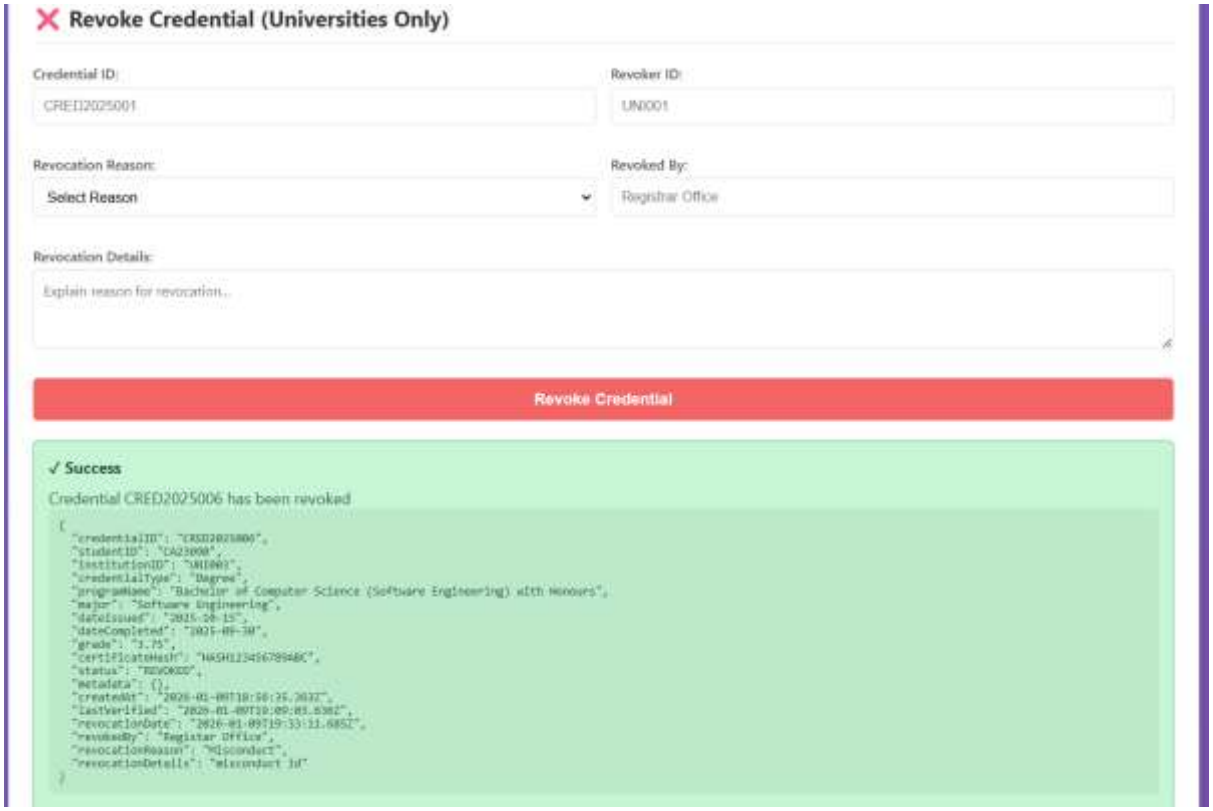


Figure 56 : Revocation Success - Frontend Response

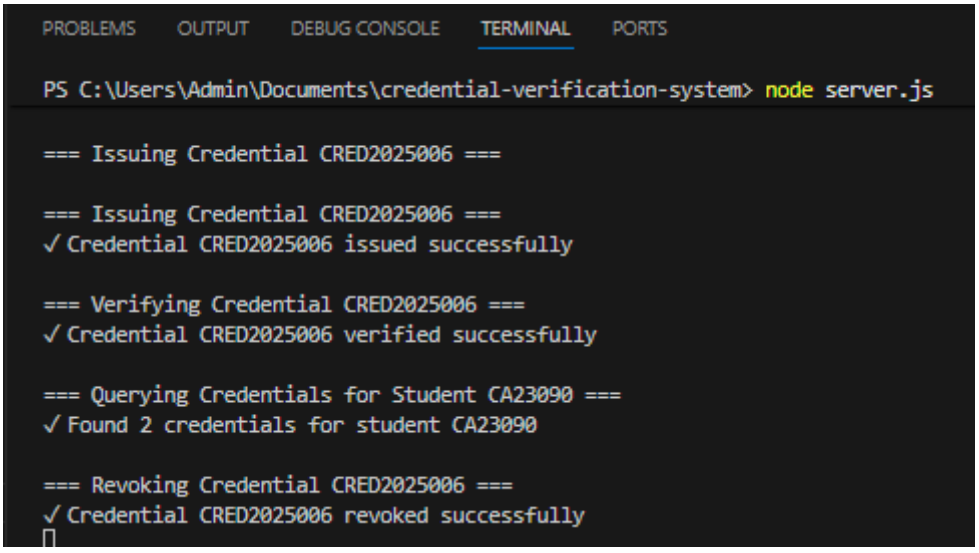


Figure 57: Server Console - Revocation Processing

```
"CRED2025006": {
  "credentialID": "CRED2025006",
  "studentID": "CA23090",
  "institutionID": "UNI003",
  "credentialType": "Degree",
  "programName": "Bachelor of Computer Science (Software Engineering) with Honours",
  "major": "Software Engineering",
  "dateIssued": "2025-10-15",
  "dateCompleted": "2025-09-30",
  "grade": "3.75",
  "certificateHash": "HASH123456789ABC",
  "status": "REVOKED",
  "metadata": {},
  "createdAt": "2026-01-09T18:56:35.363Z",
  "lastVerified": "2026-01-09T19:09:03.636Z",
  "revocationDate": "2026-01-09T19:33:11.685Z",
  "revokedBy": "Registar Office",
  "revocationReason": "Misconduct",
  "revocationDetails": "misconduct id"
```


Figure 58: Ledger Update - Revocation Metadata Added

Justification:

The testing and validation process comprehensively demonstrates that the Blockchain-Based Academic Credential Verification System achieves complete end-to-end integration across all components.

4.2 Smart Contract Source Code (.js file)

4.2.1 credentialContract.js

```
const fs = require('fs-extra');
const path = require('path');

// Path to ledger file
const ledgerPath = path.join(__dirname, '../ledger/ledger.json');

class CredentialContract {

    // Helper function to read ledger
    _readLedger() {
        try {
            const data = fs.readJsonSync(ledgerPath);
            return {
                credentials: data.credentials || {},
                students: data.students || {},
                institutions: data.institutions || {},
                verifications: data.verifications || []
            };
        } catch (error) {
            return { credentials: {}, students: {}, institutions: {}, verifications: [] };
        }
    }

    // Helper function to write ledger
    _writeLedger(data) {
        fs.writeJsonSync(ledgerPath, data, { spaces: 2 });
    }

    // Function 1: Issue Credential
    async issueCredential(credentialID, studentID, institutionID, credentialType, programName, major, dateIssued, dateCompleted, grade, certificateHash, metadata) {
        console.log(`\n=== Issuing Credential ${credentialID} ===`);

        // Read current ledger
        let ledger = this._readLedger();

        // Validation 1: Check if credential ID already exists
        if (ledger.credentials[credentialID]) {
            throw new Error(`Credential ${credentialID} already exists`);
        }

        // Validation 2: Check if student exists
        if (!ledger.students[studentID]) {
            throw new Error(`Student ${studentID} not found`);
        }

        // Validation 3: Check if institution exists
        if (!ledger.institutions[institutionID]) {
            throw new Error(`Institution ${institutionID} not found`);
        }

        // Validation 4: Check dates
        const completedDate = new Date(dateCompleted);
        const issuedDate = new Date(dateIssued);
        if (completedDate > issuedDate) {
            throw new Error('Completion date cannot be after issue date');
        }

        // Create credential object
        const credential = {
            credentialID,
            studentID,
            institutionID,
            credentialType,
            programName,
            major,
            dateIssued,
            dateCompleted,
            grade,
            certificateHash,
            status: 'ISSUED',
            metadata: metadata || {},
            createdAt: new Date().toISOString()
        };

        // Store in ledger
        ledger.credentials[credentialID] = credential;
        this._writeLedger(ledger);

        console.log(`✓ Credential ${credentialID} issued successfully`);
        return { success: true, message: `Credential ${credentialID} issued successfully`, credential };
    }
```

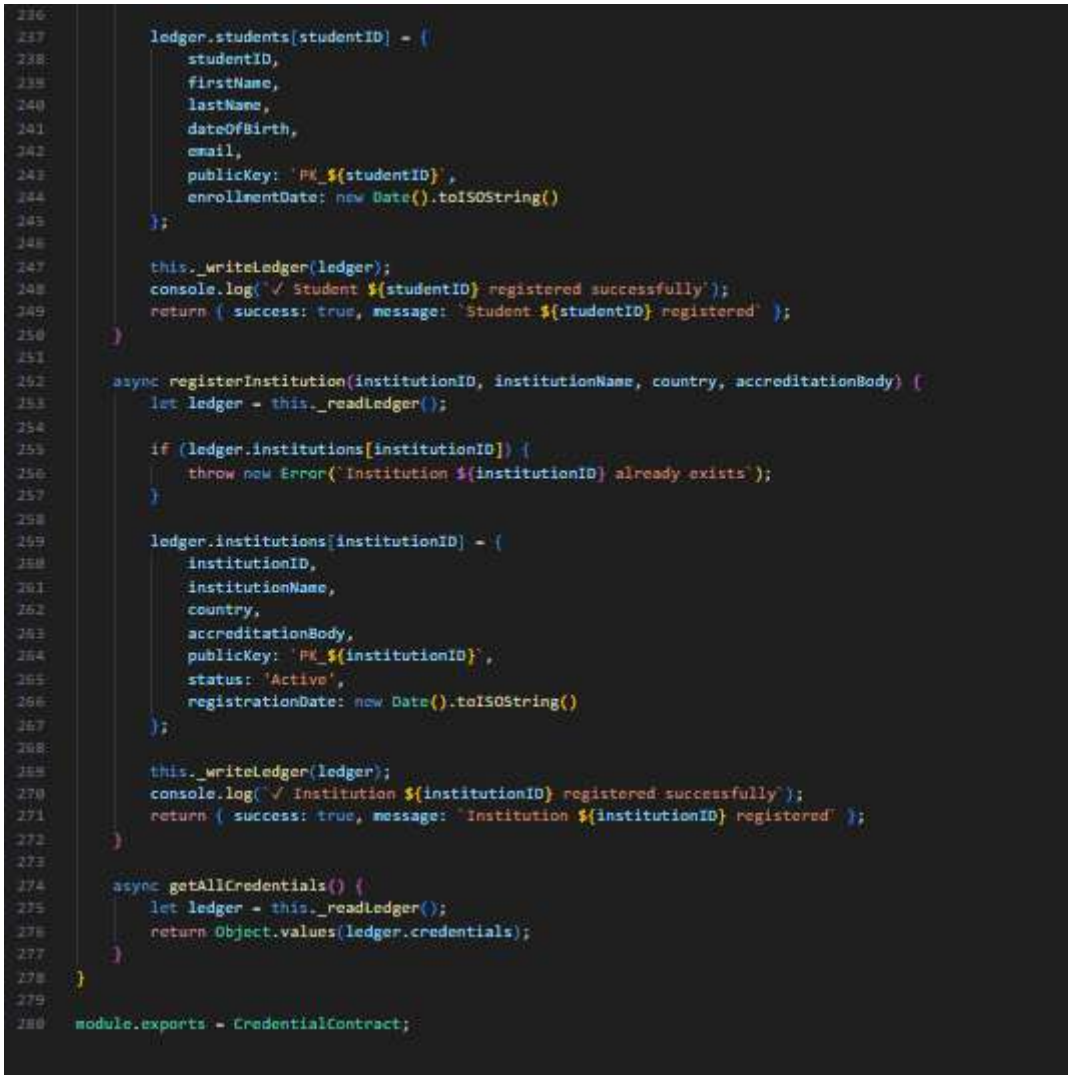
Figure 59: credentialContract.js code

```
    }

    // Function 2: Verify Credential
    async verifyCredential(credentialID, requesterID, requesterType, studentConsent) {
        console.log(`\n=== Verifying Credential ${credentialID} ===`);

        let ledger = this._readLedger();

        // Validation 1: Check if credential exists
        if (!ledger.credentials[credentialID]) {
            throw new Error(`Credential ${credentialID} not found`);
        }

        // Validation 2: Check student consent
        if (!studentConsent) {
            throw new Error('Student consent required for verification');
        }

        const credential = ledger.credentials[credentialID];

        // Validation 3: Check if credential is revoked
        if (credential.status === 'REVOKED') {
            return {
                success: false,
                message: 'Credential has been revoked',
                verificationResult: false
            };
        }

        // Create verification record
        const verificationRequest = {
            requestID: `REQ${Date.now()}`,
            credentialID,
            requesterID,
            requesterType,
            requestDate: new Date().toISOString(),
            verificationResult: true,
            accessGranted: studentConsent
        };

        // Add to verification history
        ledger.verifications.push(verificationRequest);

        // Update credential status to VERIFIED
        credential.status = 'VERIFIED';
        credential.lastVerified = new Date().toISOString();

        this._writeLedger(ledger);

        console.log(`✓ Credential ${credentialID} verified successfully`);
        return {
            success: true,
            message: 'Credential is valid',
            verificationResult: true,
            credential,
            verificationRequest
        };
    }

    // Function 3: Revoke Credential
    async revokeCredential(credentialID, revokerID, revocationReason, revocationDetails, revokedBy) {
        console.log(`\n=== Revoking Credential ${credentialID} ===`);

        let ledger = this._readLedger();

        // Validation 1: Check if credential exists
        if (!ledger.credentials[credentialID]) {
            throw new Error(`Credential ${credentialID} not found`);
        }

        const credential = ledger.credentials[credentialID];

        // Validation 2: Check if already revoked
        if (credential.status === 'REVOKED') {
            throw new Error('Credential is already revoked');
        }
```

Figure 60: credentialContract.js code

```
        // Validation 3: Check if revoker is the issuing institution
        if (credential.institutionID !== revokerID) {
            throw new Error('Only issuing institution can revoke credential');
        }

        // Validation 4: Check revocation reason
        const validReasons = ['Fraud', 'Error', 'Misconduct', 'Other'];
        if (!validReasons.includes(revocationReason)) {
            throw new Error('Invalid revocation reason');
        }

        // Update credential
        credential.status = 'REVOKED';
        credential.revocationDate = new Date().toISOString();
        credential.revokedBy = revokedBy;
        credential.revocationReason = revocationReason;
        credential.revocationDetails = revocationDetails;

        this._writeLedger(ledger);

        console.log(`✓ Credential ${credentialID} revoked successfully`);
        return {
            success: true,
            message: `Credential ${credentialID} has been revoked`,
            credential
        };
    }

    // Function 4: Query Credentials by Student
    async queryCredentialsByStudent(studentID, requesterID, includeRevoked = false) {
        console.log(`\n=== Querying Credentials for Student ${studentID} ===`);

        let ledger = this._readLedger();

        // Validation: Check if student exists
        if (!ledger.students[studentID]) {
            throw new Error(`Student ${studentID} not found`);
        }

        // Find all credentials for this student
        const studentCredentials = Object.values(ledger.credentials).filter(cred => {
            if (cred.studentID === studentID) {
                // Filter based on includeRevoked flag
                if (!includeRevoked && cred.status === 'REVOKED') {
                    return false;
                }
                return true;
            }
            return false;
        });

        // Sort by date issued (most recent first)
        studentCredentials.sort((a, b) => new Date(b.dateIssued) - new Date(a.dateIssued));

        // Calculate statistics
        const stats = {
            totalCredentials: studentCredentials.length,
            activeCredentials: studentCredentials.filter(c => c.status === 'ISSUED' || c.status === 'VERIFIED').length,
            revokedCredentials: studentCredentials.filter(c => c.status === 'REVOKED').length
        };

        console.log(`✓ Found ${studentCredentials.length} credentials for student ${studentID}`);
        return {
            success: true,
            credentials: studentCredentials,
            statistics: stats,
            student: ledger.students[studentID]
        };
    }

    // Helper Functions for Testing

    async registerStudent(studentID, firstName, lastName, dateOfBirth, email) {
        let ledger = this._readLedger();

        if (ledger.students[studentID]) {
            throw new Error(`Student ${studentID} already exists`);
        }
```

Figure 61: credentialContract.js code



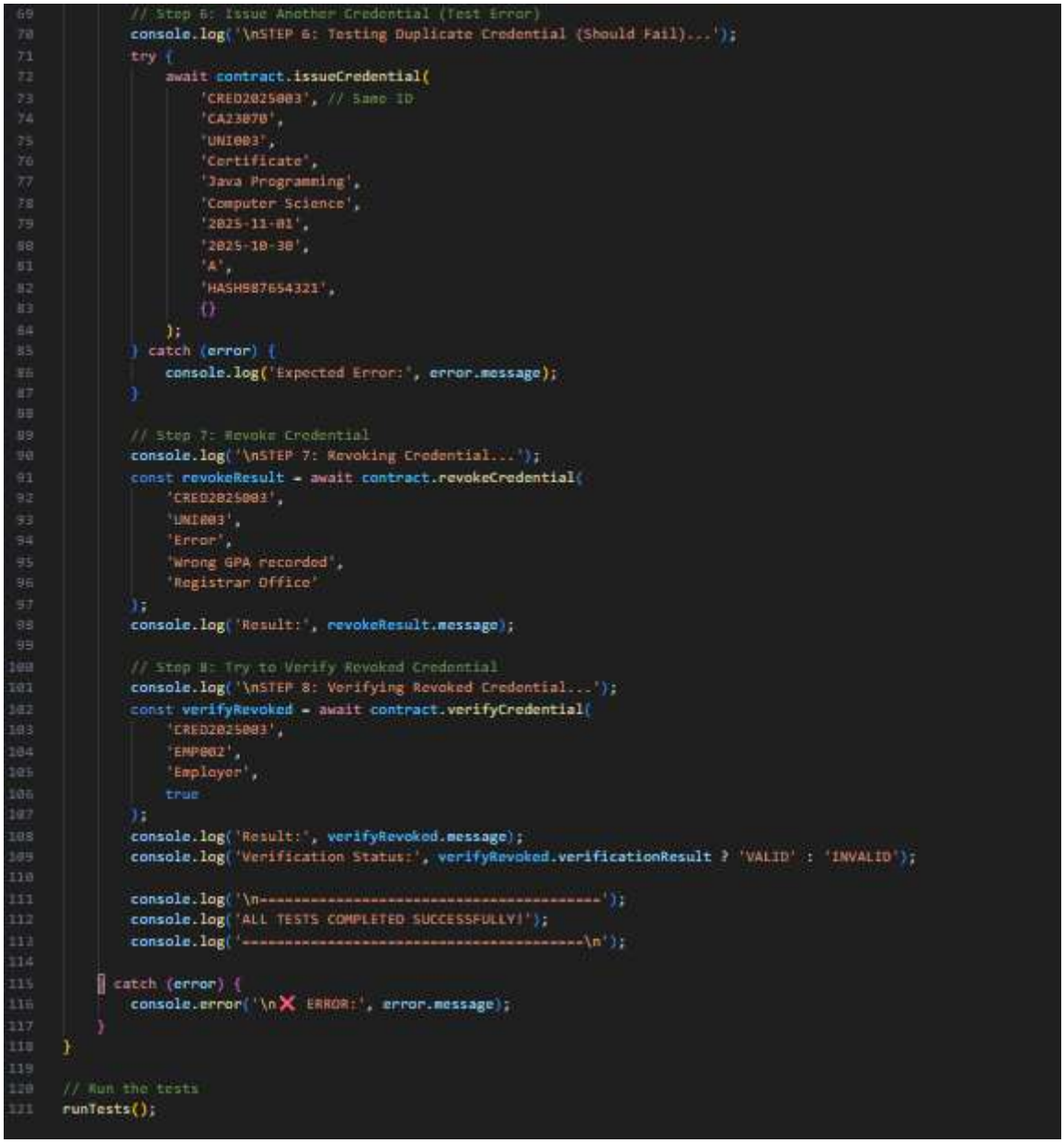
```
        ledger.students[studentID] = {
            studentID,
            firstName,
            lastName,
            dateOfBirth,
            email,
            publicKey: `PK_${studentID}`,
            enrollmentDate: new Date().toISOString()
        };

        this._writeLedger(ledger);
        console.log(`✓ Student ${studentID} registered successfully`);
        return { success: true, message: `Student ${studentID} registered` };
    }

    async registerInstitution(institutionID, institutionName, country, accreditationBody) {
        let ledger = this._readLedger();

        if (ledger.institutions[institutionID]) {
            throw new Error(`Institution ${institutionID} already exists`);
        }

        ledger.institutions[institutionID] = {
            institutionID,
            institutionName,
            country,
            accreditationBody,
            publicKey: `PK_${institutionID}`,
            status: 'Active',
            registrationDate: new Date().toISOString()
        };

        this._writeLedger(ledger);
        console.log(`✓ Institution ${institutionID} registered successfully`);
        return { success: true, message: `Institution ${institutionID} registered` };
    }

    async getAllCredentials() {
        let ledger = this._readLedger();
        return Object.values(ledger.credentials);
    }
}

module.exports = CredentialContract;
```

Figure 62: credentialContract.js code

4.2.2 testContract.js

```
const CredentialContract = require('../contracts/credentialContract');

async function runTests() {
    const contract = new CredentialContract();

    console.log('\n=====================================');
    console.log('ACADEMIC CREDENTIAL VERIFICATION SYSTEM');
    console.log('Smart Contract Testing');
    console.log('=====================================\n');

    try {
        // Step 1: Register Institution
        console.log('STEP 1: Registering Institution...');
        await contract.registerInstitution(
            'UNI003',
            'Universiti Malaysia Kelantan',
            'Malaysia',
            'Malaysian Qualifications Agency'
        );

        // Step 2: Register Student
        console.log('\nSTEP 2: Registering Student...');
        await contract.registerStudent(
            'CA23070',
            'Muhammad Ali bin',
            'Syakir',
            '2001-05-15',
            'ca23070@student.umpsa.edu.my'
        );

        // Step 3: Issue Credential
        console.log('\nSTEP 3: Issuing Credential...');
        const issueResult = await contract.issueCredential(
            'CRED2025003',
            'CA23070',
            'UNI003',
            'Degree',
            'Bachelor of Computer Science (Software Engineering) with Honours',
            'Software Engineering',
            '2025-10-15',
            '2025-09-30',
            '3.75',
            'HASH123456789ABC',
            { honors: 'Second Class Upper', dean: 'Prof. Dr. Ahmad' }
        );
        console.log('Result:', issueResult.message);

        // Step 4: Verify Credential
        console.log('\nSTEP 4: Verifying Credential...');
        const verifyResult = await contract.verifyCredential(
            'CRED2025003',
            'EMP001',
            'Employer',
            true
        );
        console.log('Result:', verifyResult.message);
        console.log('Verification Status:', verifyResult.verificationResult ? 'VALID' : 'INVALID');

        // Step 5: Query Student Credentials
        console.log('\nSTEP 5: Querying Student Credentials...');
        const queryResult = await contract.queryCredentialsByStudent(
            'CA23070',
            'CA23070',
            false
        );
        console.log('Result: Found', queryResult.credentials.length, 'credential(s)');
        console.log('Statistics:', queryResult.statistics);
```

Figure 63: testContract.js code



```
        // Step 6: Issue Another Credential (Test Error)
        console.log('\nSTEP 6: Testing Duplicate Credential (Should Fail)...');
        try {
            await contract.issueCredential(
                'CRED2025003', // Same ID
                'CA23070',
                'UNI003',
                'Certificate',
                'Java Programming',
                'Computer Science',
                '2025-11-01',
                '2025-10-30',
                'A',
                'HASH987654321',
                {}
            );
        } catch (error) {
            console.log('Expected Error:', error.message);
        }

        // Step 7: Revoke Credential
        console.log('\nSTEP 7: Revoking Credential...');
        const revokeResult = await contract.revokeCredential(
            'CRED2025003',
            'UNI003',
            'Error',
            'Wrong GPA recorded',
            'Registrar Office'
        );
        console.log('Result:', revokeResult.message);

        // Step 8: Try to Verify Revoked Credential
        console.log('\nSTEP 8: Verifying Revoked Credential...');
        const verifyRevoked = await contract.verifyCredential(
            'CRED2025003',
            'EMP002',
            'Employer',
            true
        );
        console.log('Result:', verifyRevoked.message);
        console.log('Verification Status:', verifyRevoked.verificationResult ? 'VALID' : 'INVALID');

        console.log('\n----------------------------------------');
        console.log('ALL TESTS COMPLETED SUCCESSFULLY!');
        console.log('----------------------------------------\n');

    } catch (error) {
        console.error('\n❌ ERROR:', error.message);
    }
}

// Run the tests
runTests();
```

Figure 64: testContract.js code

4.3 Client Application Code - HTML file-codes (.html file) and JavaScript (.js file)

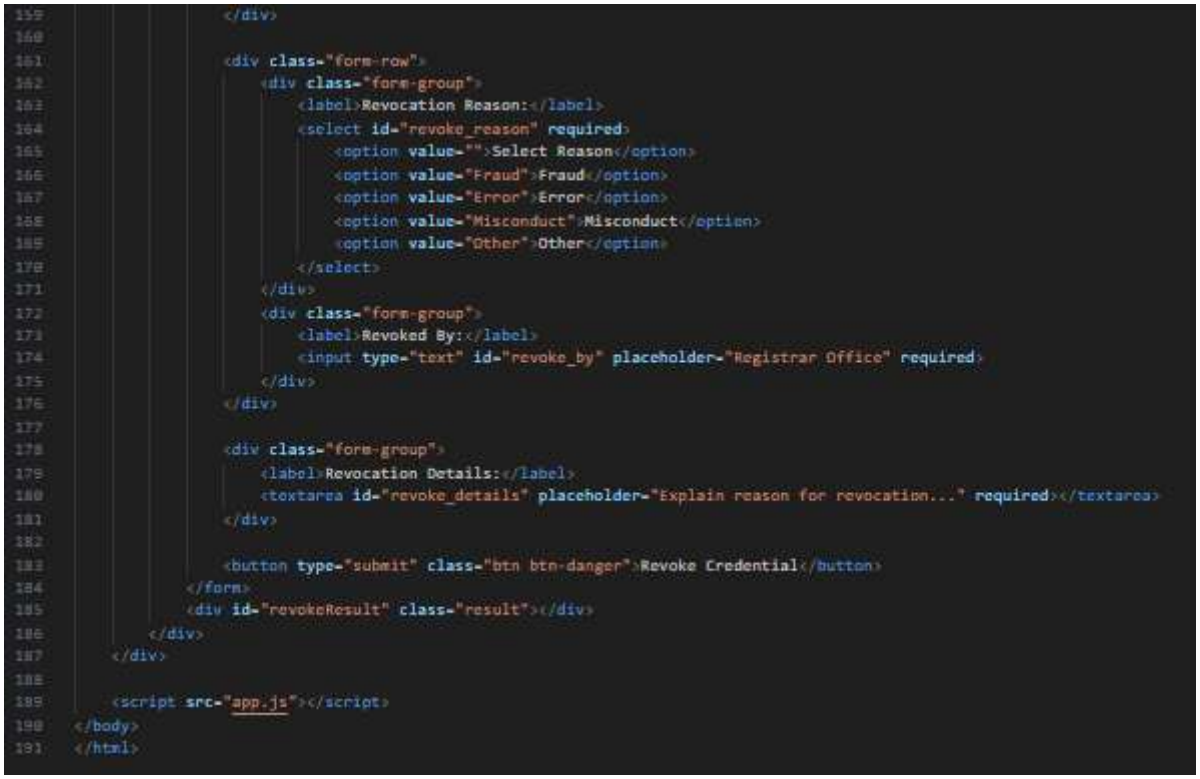
4.3.1 Index.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Academic Credential Verification System</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <header>
            <h1>🎓 Academic Credential Verification System</h1>
            <p>Blockchain-Based Credential Management</p>
        </header>

        <!-- Section 1: Issue Credential -->
        <div class="card">
            <h2>📄 Issue Credential (Universities Only)</h2>
            <form id="issueForm">
                <div class="form-row">
                    <div class="form-group">
                        <label>Credential ID:</label>
                        <input type="text" id="issue_credID" placeholder="CRED2025001" required>
                    </div>
                    <div class="form-group">
                        <label>Student ID:</label>
                        <input type="text" id="issue_studentID" placeholder="CA23085" required>
                    </div>
                </div>

                <div class="form-row">
                    <div class="form-group">
                        <label>Institution ID:</label>
                        <input type="text" id="issue_institutionID" placeholder="UNI001" required>
                    </div>
                    <div class="form-group">
                        <label>Credential Type:</label>
                        <select id="issue_type" required>
                            <option value="">Select Type</option>
                            <option value="Degree">Degree</option>
                            <option value="Diploma">Diploma</option>
                            <option value="Certificate">Certificate</option>
                        </select>
                    </div>
                </div>

                <div class="form-group">
                    <label>Program Name:</label>
                    <input type="text" id="issue_program" placeholder="Bachelor of Computer Science" required>
                </div>

                <div class="form-row">
                    <div class="form-group">
                        <label>Major:</label>
                        <input type="text" id="issue_major" placeholder="Software Engineering" required>
                    </div>
                    <div class="form-group">
                        <label>Grade/CGPA:</label>
                        <input type="text" id="issue_grade" placeholder="3.75" required>
                    </div>
                </div>

                <div class="form-row">
                    <div class="form-group">
                        <label>Date Completed:</label>
                        <input type="date" id="issue_dateCompleted" required>
                    </div>
                    <div class="form-group">
                        <label>Date Issued:</label>
                        <input type="date" id="issue_dateIssued" required>
                    </div>
                </div>

                <div class="form-group">
                    <label>Certificate Hash:</label>
                    <input type="text" id="issue_hash" placeholder="HASH123456789ABC" required>
                </div>

                <button type="submit" class="btn btn-primary">Issue Credential</button>
            </form>
            <div id="issueResult" class="result"></div>
```

Figure 65: Index.html code

```
        </div>

        <!-- Section 2: Verify Credential -->
        <div class="card">
            <h2>✅ Verify Credential (Employers/Institutions)</h2>
            <form id="verifyForm">
                <div class="form-row">
                    <div class="form-group">
                        <label>Credential ID:</label>
                        <input type="text" id="verify_credID" placeholder="CRED2025001" required>
                    </div>
                    <div class="form-group">
                        <label>Requester ID:</label>
                        <input type="text" id="verify_requesterID" placeholder="EMP001" required>
                    </div>
                </div>

                <div class="form-row">
                    <div class="form-group">
                        <label>Requester Type:</label>
                        <select id="verify_type" required>
                            <option value="">Select Type</option>
                            <option value="Employer">Employer</option>
                            <option value="Institution">Institution</option>
                            <option value="Individual">Individual</option>
                        </select>
                    </div>
                    <div class="form-group">
                        <label>Student Consent:</label>
                        <select id="verify_consent" required>
                            <option value="true">Yes (Granted)</option>
                            <option value="false">No (Denied)</option>
                        </select>
                    </div>
                </div>

                <button type="submit" class="btn btn-success">Verify Credential</button>
            </form>
            <div id="verifyResult" class="result"></div>
        </div>

        <!-- Section 3: Query Student Credentials -->
        <div class="card">
            <h2>🔍 Query Student Credentials</h2>
            <form id="queryForm">
                <div class="form-row">
                    <div class="form-group">
                        <label>Student ID:</label>
                        <input type="text" id="query_studentID" placeholder="CA23085" required>
                    </div>
                    <div class="form-group">
                        <label>Include Revoked:</label>
                        <select id="query_includeRevoked">
                            <option value="false">No (Active Only)</option>
                            <option value="true">Yes (Include Revoked)</option>
                        </select>
                    </div>
                </div>

                <button type="submit" class="btn btn-info">Query Credentials</button>
            </form>
            <div id="queryResult" class="result"></div>
        </div>

        <!-- Section 4: Revoke Credential -->
        <div class="card">
            <h2>❌ Revoke Credential (Universities Only)</h2>
            <form id="revokeForm">
                <div class="form-row">
                    <div class="form-group">
                        <label>Credential ID:</label>
                        <input type="text" id="revoke_credID" placeholder="CRED2025001" required>
                    </div>
                    <div class="form-group">
                        <label>Revoker ID:</label>
                        <input type="text" id="revoke_revokerID" placeholder="UNI001" required>
                    </div>
```

Figure 66 : Index.html code



```
                </div>

                <div class="form-row">
                    <div class="form-group">
                        <label>Revocation Reason:</label>
                        <select id="revoke_reason" required>
                            <option value="">Select Reason</option>
                            <option value="Fraud">Fraud</option>
                            <option value="Error">Error</option>
                            <option value="Misconduct">Misconduct</option>
                            <option value="Other">Other</option>
                        </select>
                    </div>
                    <div class="form-group">
                        <label>Revoked By:</label>
                        <input type="text" id="revoke_by" placeholder="Registrar Office" required>
                    </div>
                </div>

                <div class="form-group">
                    <label>Revocation Details:</label>
                    <textarea id="revoke_details" placeholder="Explain reason for revocation..." required></textarea>
                </div>

                <button type="submit" class="btn btn-danger">Revoke Credential</button>
            </form>
            <div id="revokeResult" class="result"></div>
        </div>
    </div>

    <script src="app.js"></script>
</body>
</html>
```

Figure 67: Index.html code

4.3.2 App.js

```
const API_URL = 'http://localhost:3000/api';

// Helper function to show results
function showResult(elementId, success, message, data = null) {
    const resultDiv = document.getElementById(elementId);
    resultDiv.className = `result show ${success ? 'success' : 'error'}`;

    let html = `<h3>${success ? '✓ Success' : '✗ Error'}</h3>`;
    html += `<p>${message}</p>`;

    if (data) {
        html += `<pre>${JSON.stringify(data, null, 2)}</pre>`;
    }

    resultDiv.innerHTML = html;

    // Scroll to result
    resultDiv.scrollIntoView({ behavior: 'smooth', block: 'nearest' });
}

// 1. Issue Credential
document.getElementById('issueForm').addEventListener('submit', async (e) => {
    e.preventDefault();

    const data = {
        credentialID: document.getElementById('issue_credID').value,
        studentID: document.getElementById('issue_studentID').value,
        institutionID: document.getElementById('issue_institutionID').value,
        credentialType: document.getElementById('issue_type').value,
        programName: document.getElementById('issue_program').value,
        major: document.getElementById('issue_major').value,
        grade: document.getElementById('issue_grade').value,
        dateCompleted: document.getElementById('issue_dateCompleted').value,
        dateIssued: document.getElementById('issue_dateIssued').value,
        certificateHash: document.getElementById('issue_hash').value
    };

    try {
        const response = await fetch(`${API_URL}/issue-credential`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(data)
        });

        const result = await response.json();

        if (result.success) {
            showResult('issueResult', true, result.message, result.credential);
            document.getElementById('issueForm').reset();
        } else {
            showResult('issueResult', false, result.message);
        }
    } catch (error) {
        showResult('issueResult', false, 'Network error: ' + error.message);
    }
});

// 2. Verify Credential
document.getElementById('verifyForm').addEventListener('submit', async (e) => {
    e.preventDefault();

    const data = {
        credentialID: document.getElementById('verify_credID').value,
        requesterID: document.getElementById('verify_requesterID').value,
        requesterType: document.getElementById('verify_type').value,
        studentConsent: document.getElementById('verify_consent').value
    };

    try {
        const response = await fetch(`${API_URL}/verify-credential`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(data)
        });

        const result = await response.json();

        if (result.success) {
            const verificationStatus = result.verificationResult ? 'VALID ✓' : 'INVALID ✗';
            showResult('verifyResult', result.verificationResult,
                `Credential verification: ${verificationStatus}`,
```

Figure 68: App.js code

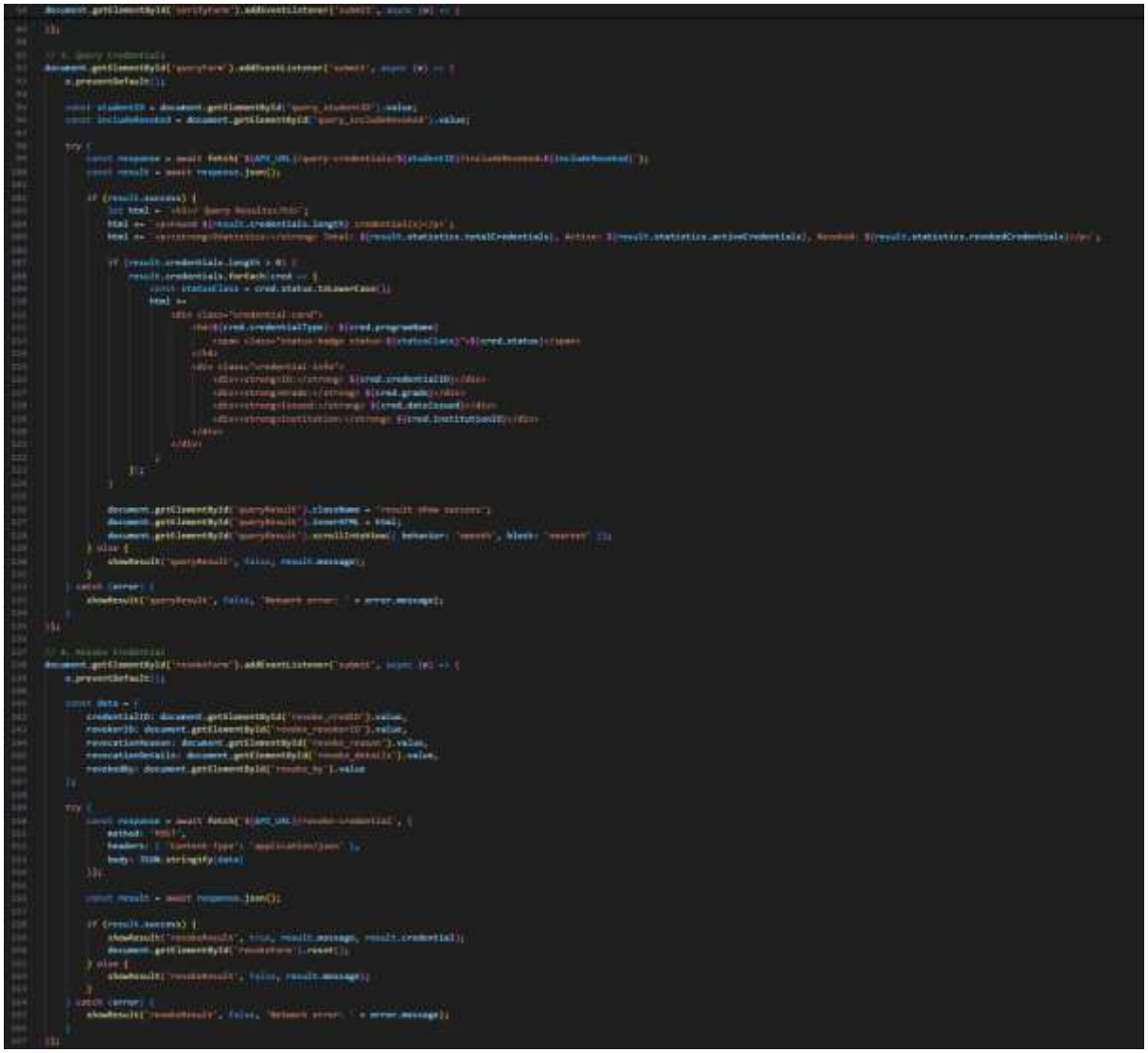


Figure 69: App.js code